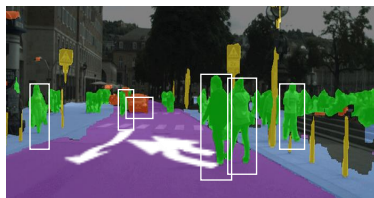
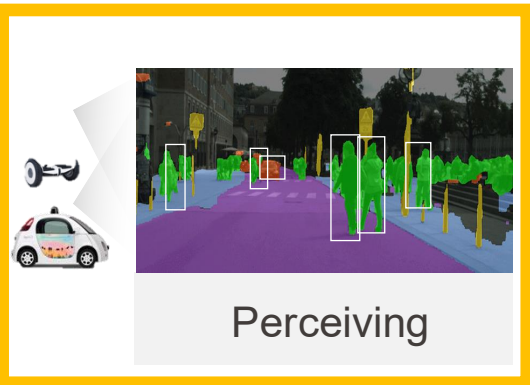


Deep Learning for Autonomous Vehicles

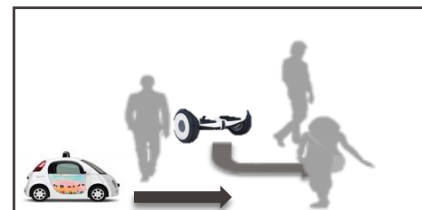
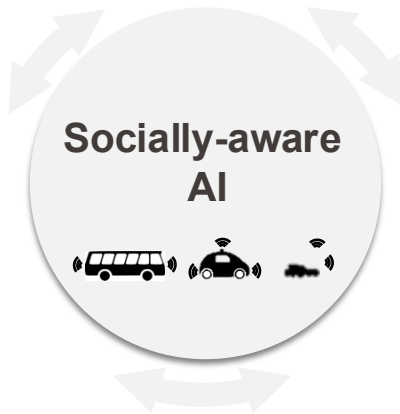
Lecture 5: Learning - Predictive models, RNN - A. Alahi



Social Forecasting



Perceiving



Planning

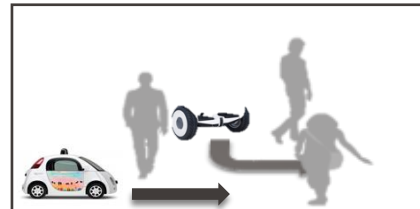


Perceiving



Social Forecasting

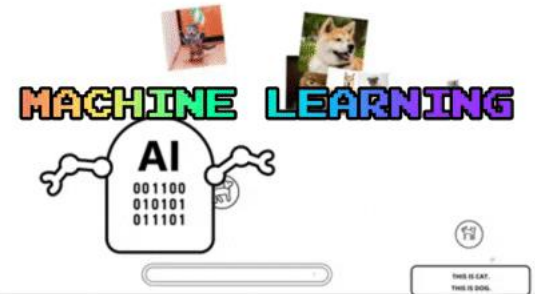
**Socially-aware
AI**



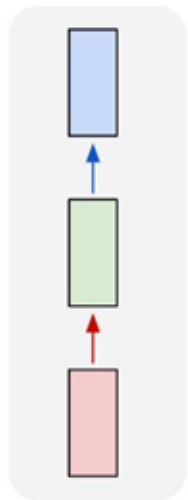
Planning

- Recurrent Neural Network

GIGABYTE™



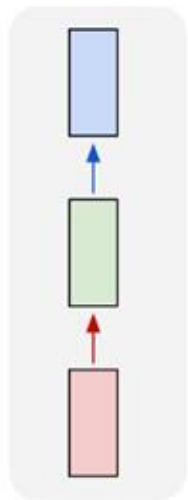
one to one



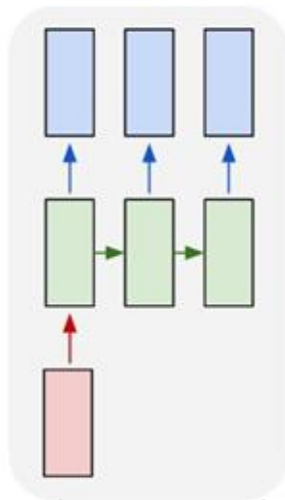
$P(y|x)$

Vanilla Neural Networks

one to one



one to many



$$P(y_1, \dots, y_t | x)$$

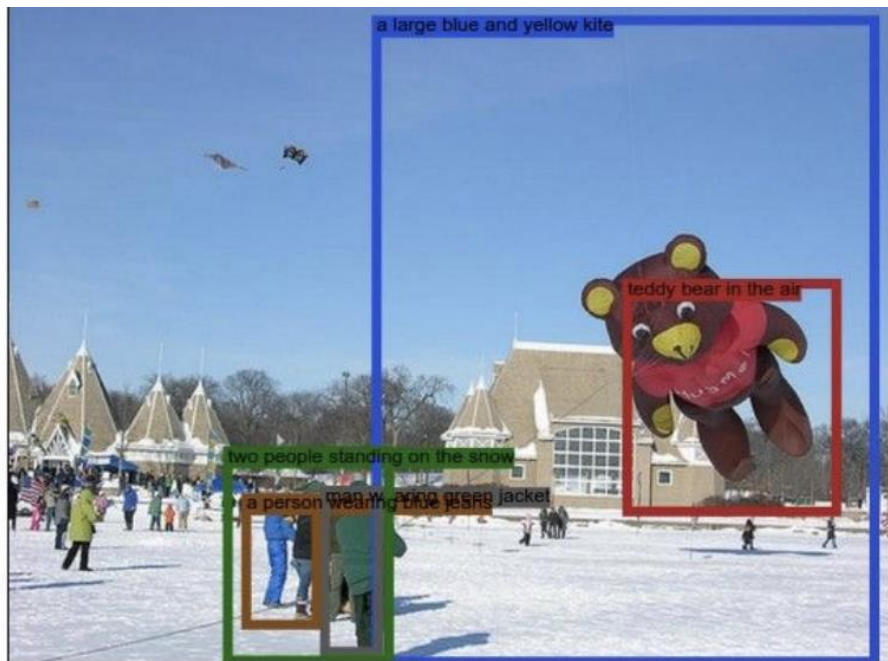
e.g., **Image Captioning**

Image $\rightarrow$ sequence of words



Image Credit: Flickr8k

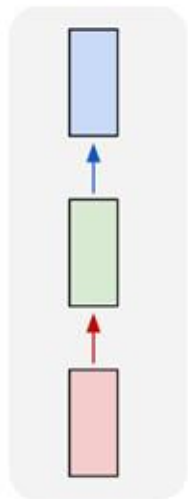
- A man is snowboarding over a structure on a snowy hill.
- A snowboarder jumps through the air on a snowy hill.
- a snowboarder wearing green pants doing a trick on a high bench
- Someone in yellow pants is on a ramp over the snow.
- The man is performing a trick on a snowboard high in the air.



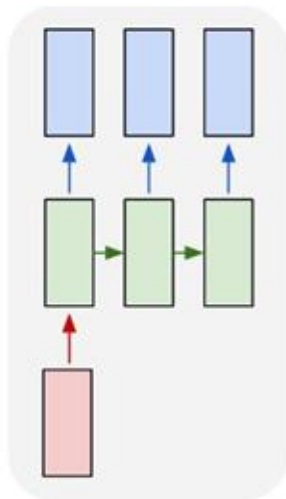
teddy bear in the air. a large blue and yellow kite.
man wearing green jacket. two people standing on
the snow. a person wearing blue jeans.

Johnson et al.

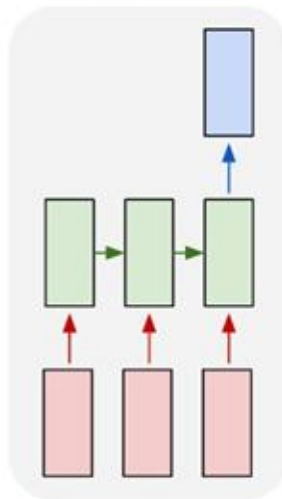
one to one



one to many



many to one

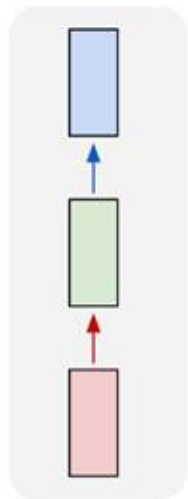


$$P(y|x_1, \dots, x_t)$$

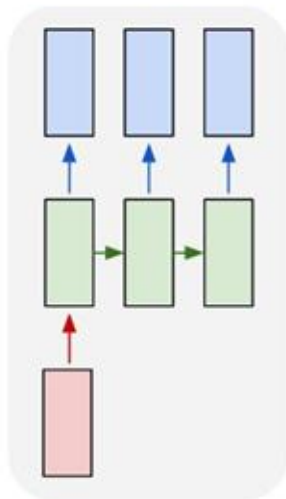
e.g., **Sentiment Classification**
sequence of words $\rightarrow$ sentiment

	Text	Positivity
0	I have bought several of the Vitality canned d...	1
1	Product arrived labeled as Jumbo Salted Peanut...	0
2	This is a confection that has been around a fe...	1
3	If you are looking for the secret ingredient i...	0
4	Great taffy at a great price. There was a wid...	1

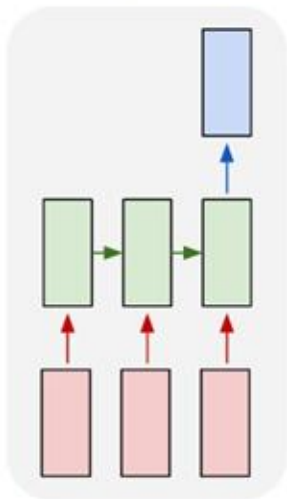
one to one



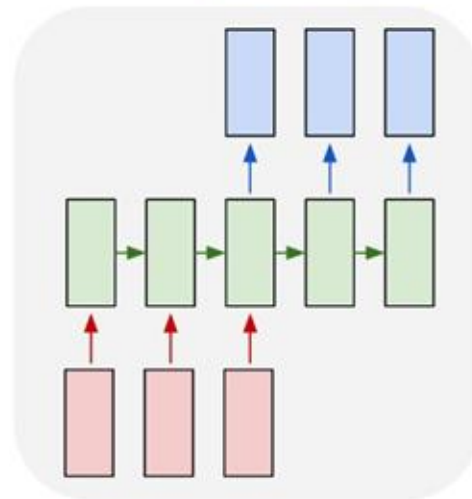
one to many



many to one

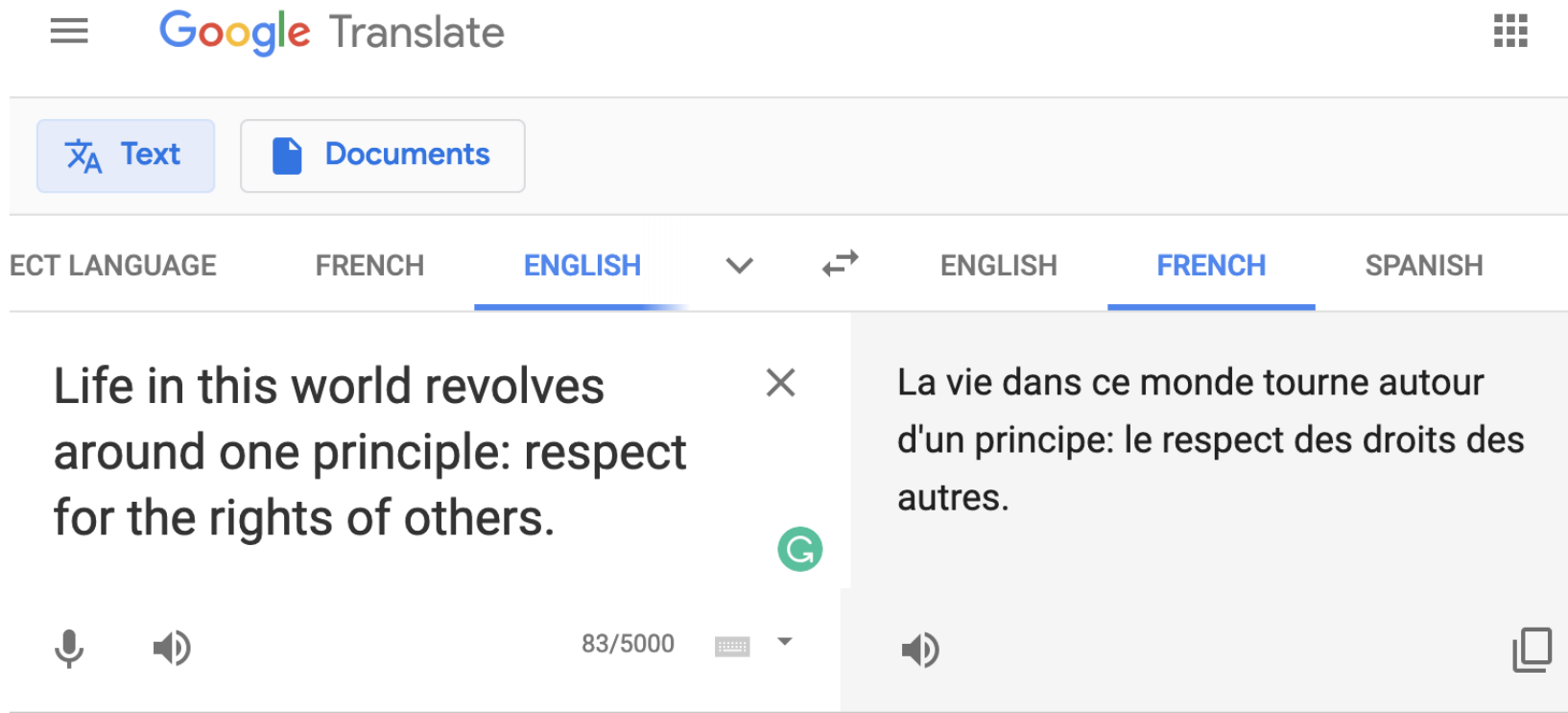


many to many



$$P(y_1, \dots, y_t | x_1, \dots, x_t)$$

e.g. **Machine Translation**
seq of words $\rightarrow$ seq of words



The screenshot shows the Google Translate web interface. At the top, there is a hamburger menu icon, the Google Translate logo, and a grid icon. Below this, there are two buttons: "Text" (with a document icon) and "Documents" (with a folder icon). The interface is divided into two main sections by a horizontal line. The left section is for the source language, and the right section is for the target language. In the left section, "ENGLISH" is selected under the "FRENCH" language group. The text "Life in this world revolves around one principle: respect for the rights of others." is entered. Below the text, there are icons for voice input, a speaker icon, and a character count "83/5000". A green circular icon with a white 'G' is also present. In the right section, "FRENCH" is selected under the "ENGLISH" language group. The translated text "La vie dans ce monde tourne autour d'un principe: le respect des droits des autres." is displayed. Below the text, there is a speaker icon and a copy icon.

Google Translate

Text Documents

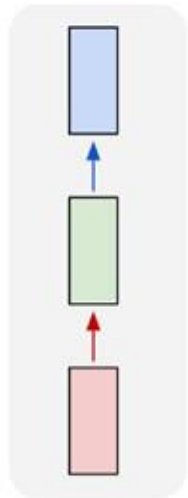
ECT LANGUAGE FRENCH **ENGLISH** ENGLISH **FRENCH** SPANISH

Life in this world revolves around one principle: respect for the rights of others.

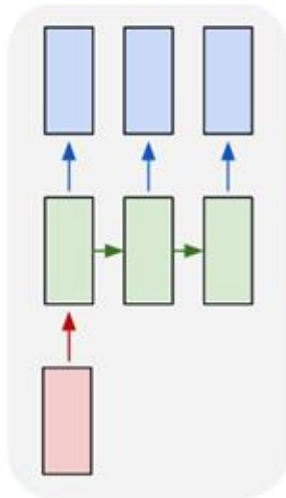
La vie dans ce monde tourne autour d'un principe: le respect des droits des autres.

83/5000

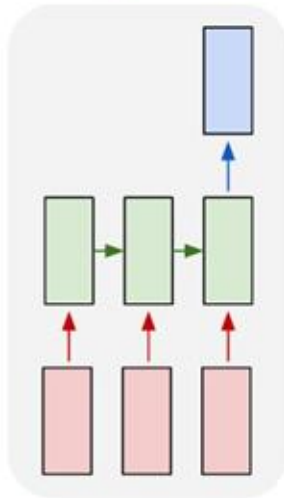
one to one



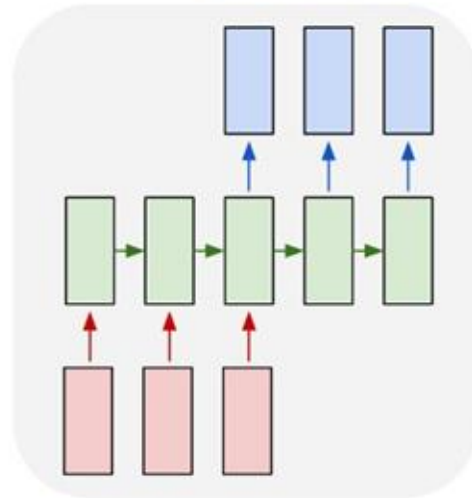
one to many



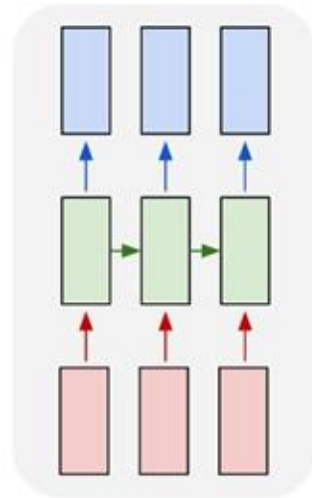
many to one



many to many

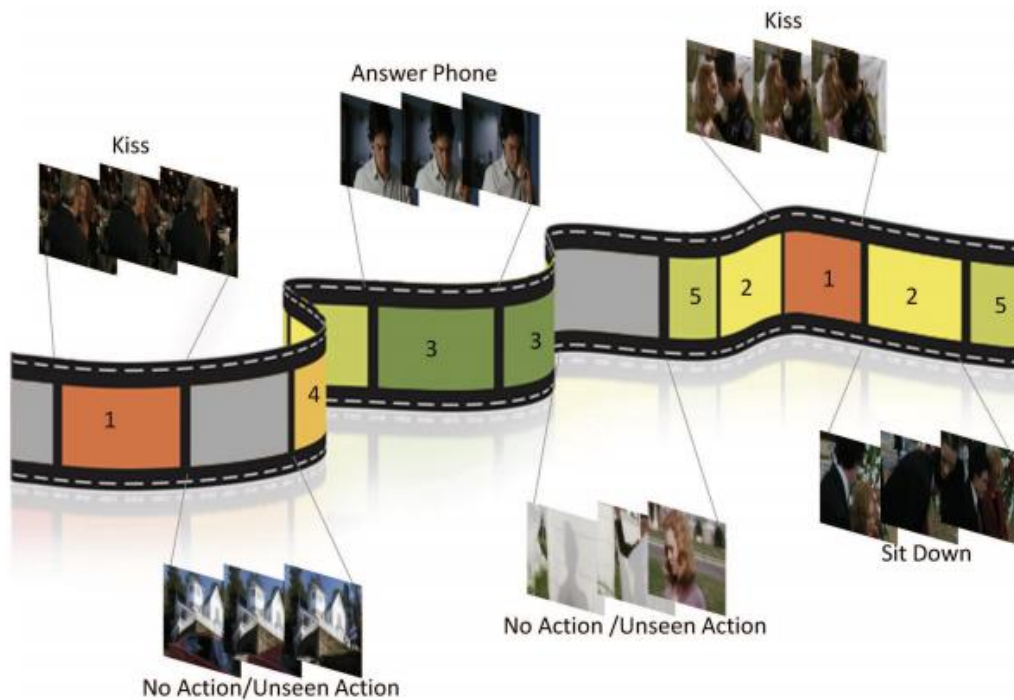


many to many



$$P(y_1, \dots, y_t | x_1, \dots, x_t)$$

e.g., Video classification on frame level



Modeling sequential data

- Chain rule to express joint distribution for a sequence of observations

$$p(\mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^T p(\mathbf{x}_t | \mathbf{x}_1, \dots, \mathbf{x}_{t-1})$$

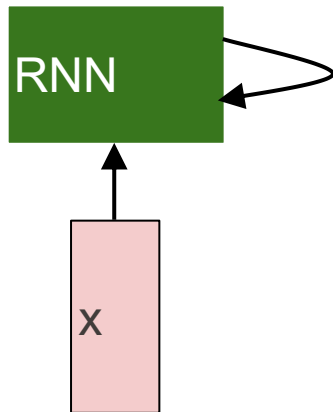
- Impractical to consider all previous observations
- Markov models assume dependence on most recent observations

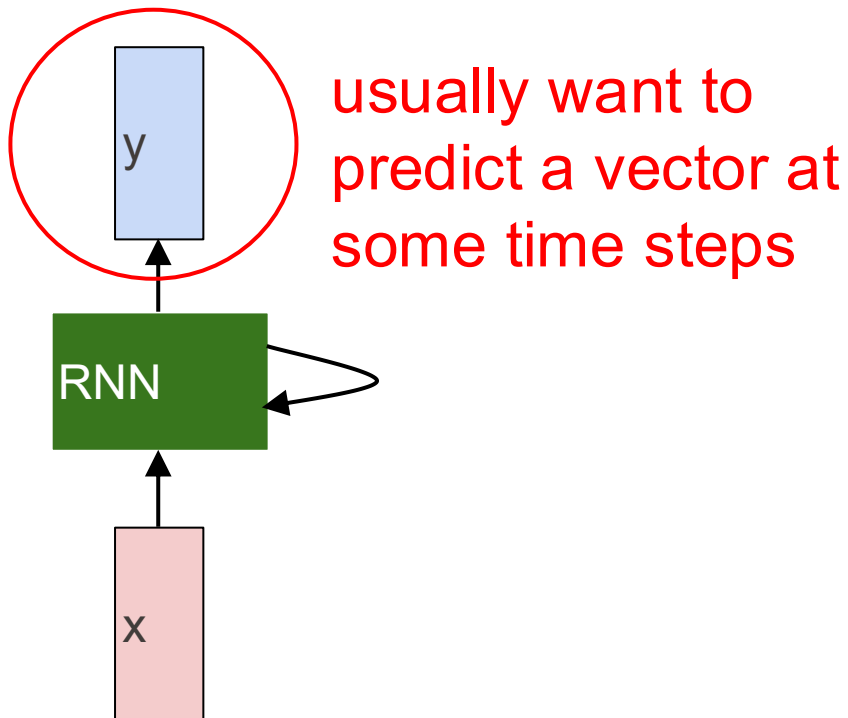
- 1st order

$$p(\mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^T p(\mathbf{x}_t | \mathbf{x}_{t-1})$$

- 2nd order

$$p(\mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^T p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_{t-2})$$





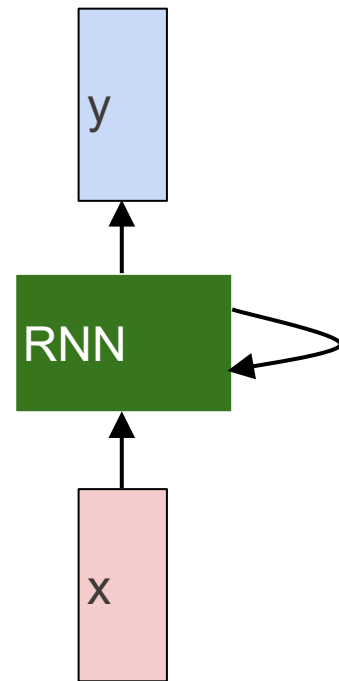
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state / old state input vector at some time step

some function with parameters W

$$y_t = W_{hy} h_t$$

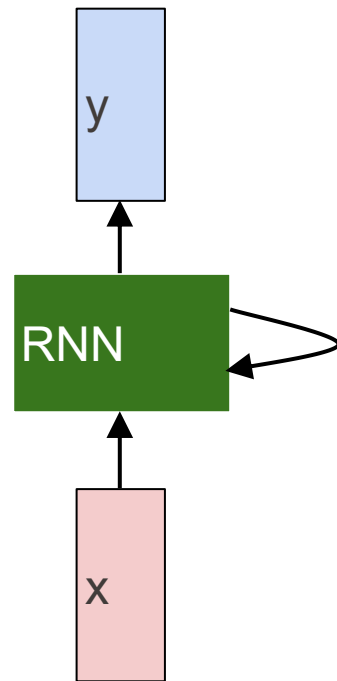


We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

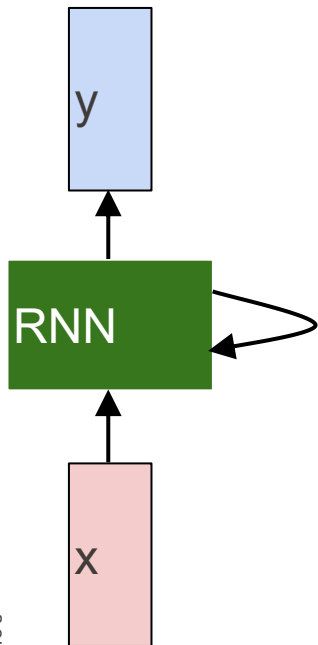
Notice: the same function and the same set of parameters are used at every time step.

$$y_t = W_{hy}h_t$$



(Vanilla) Recurrent Neural Network

The state consists of a single “hidden” vector $\mathbf{h}$:



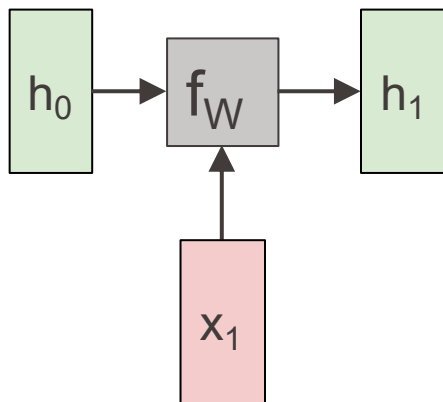
$$h_t = f_W(h_{t-1}, x_t)$$



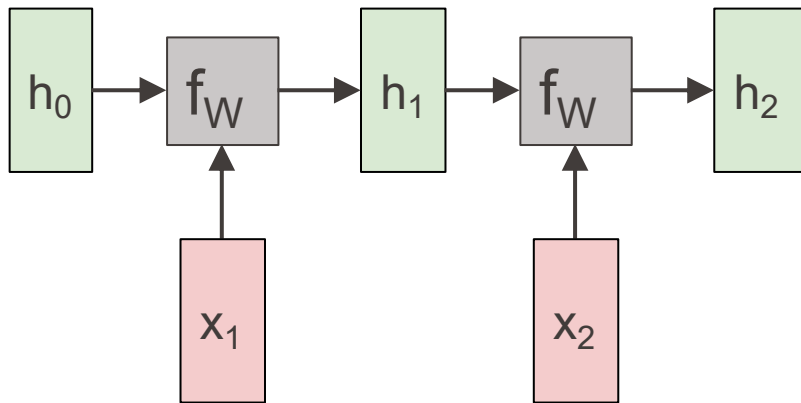
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

RNN: Computational Graph

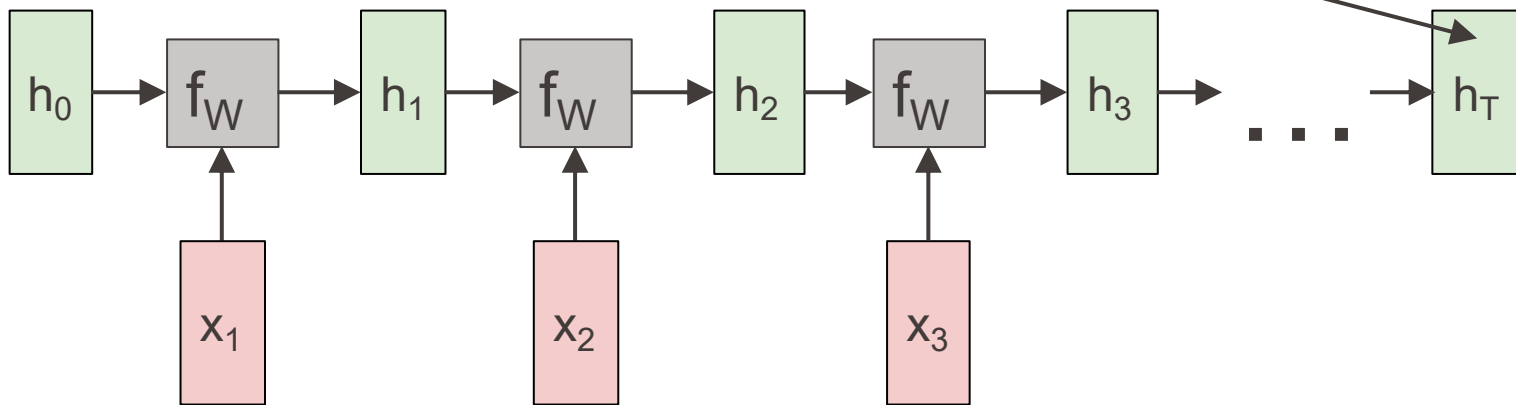


RNN: Computational Graph



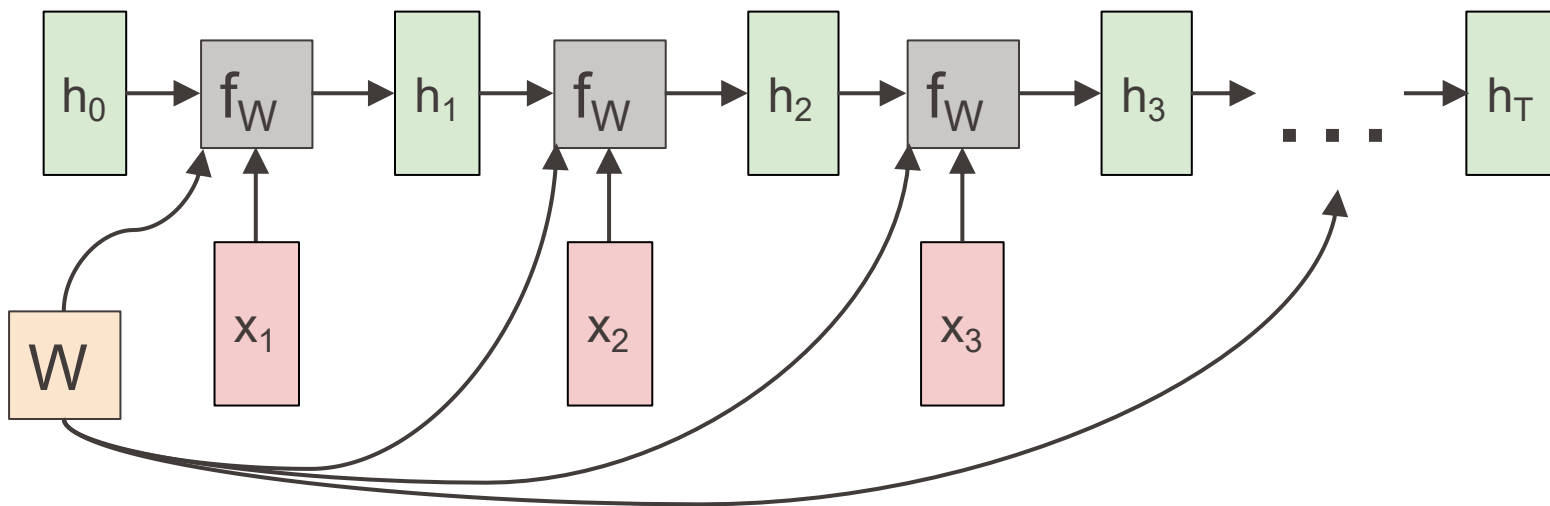
RNN: Computational Graph

h_T is a lossy summary of the past observations
It maps arbitrary length sequence $x_1 \dots x_t$ to a fixed length vector

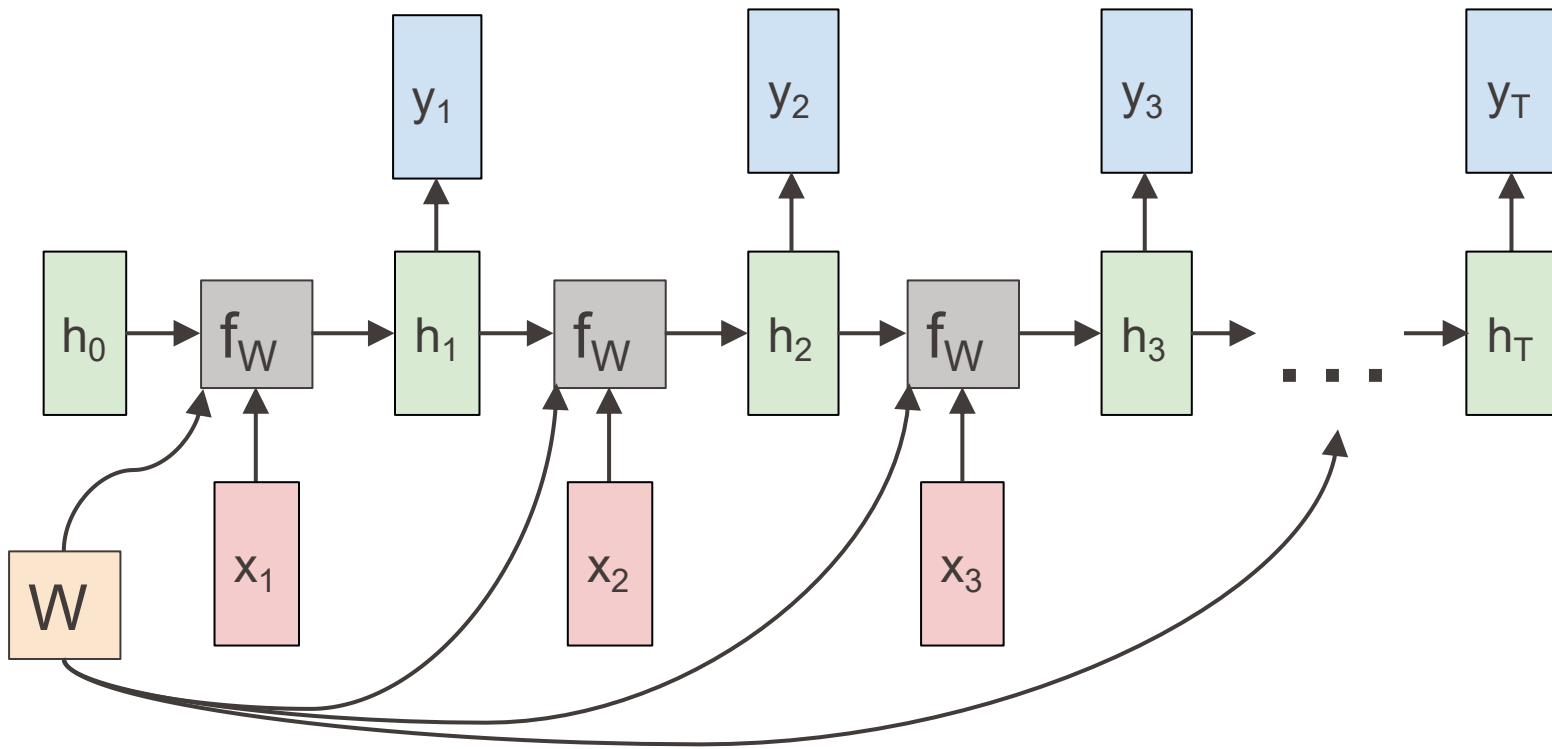


RNN: Computational Graph

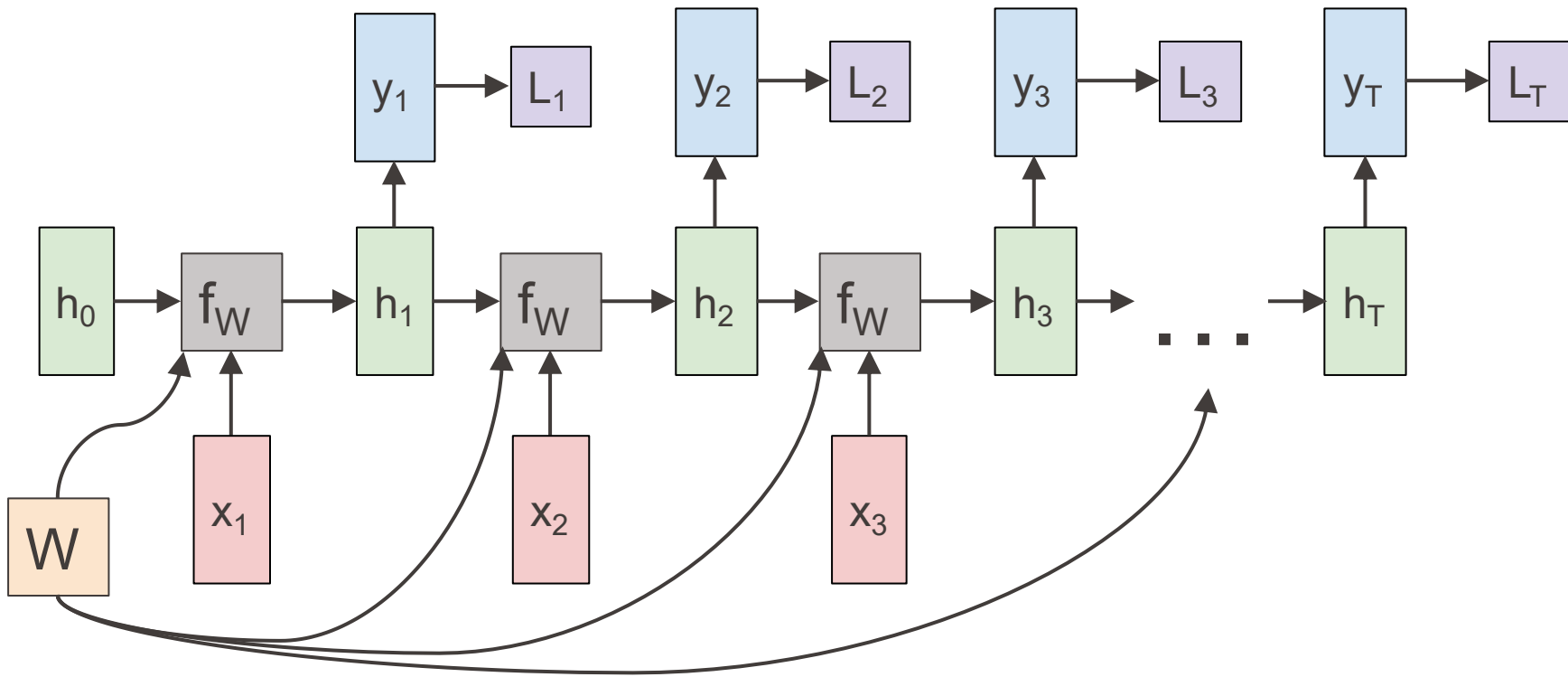
Re-use the same weight matrix at every time-step



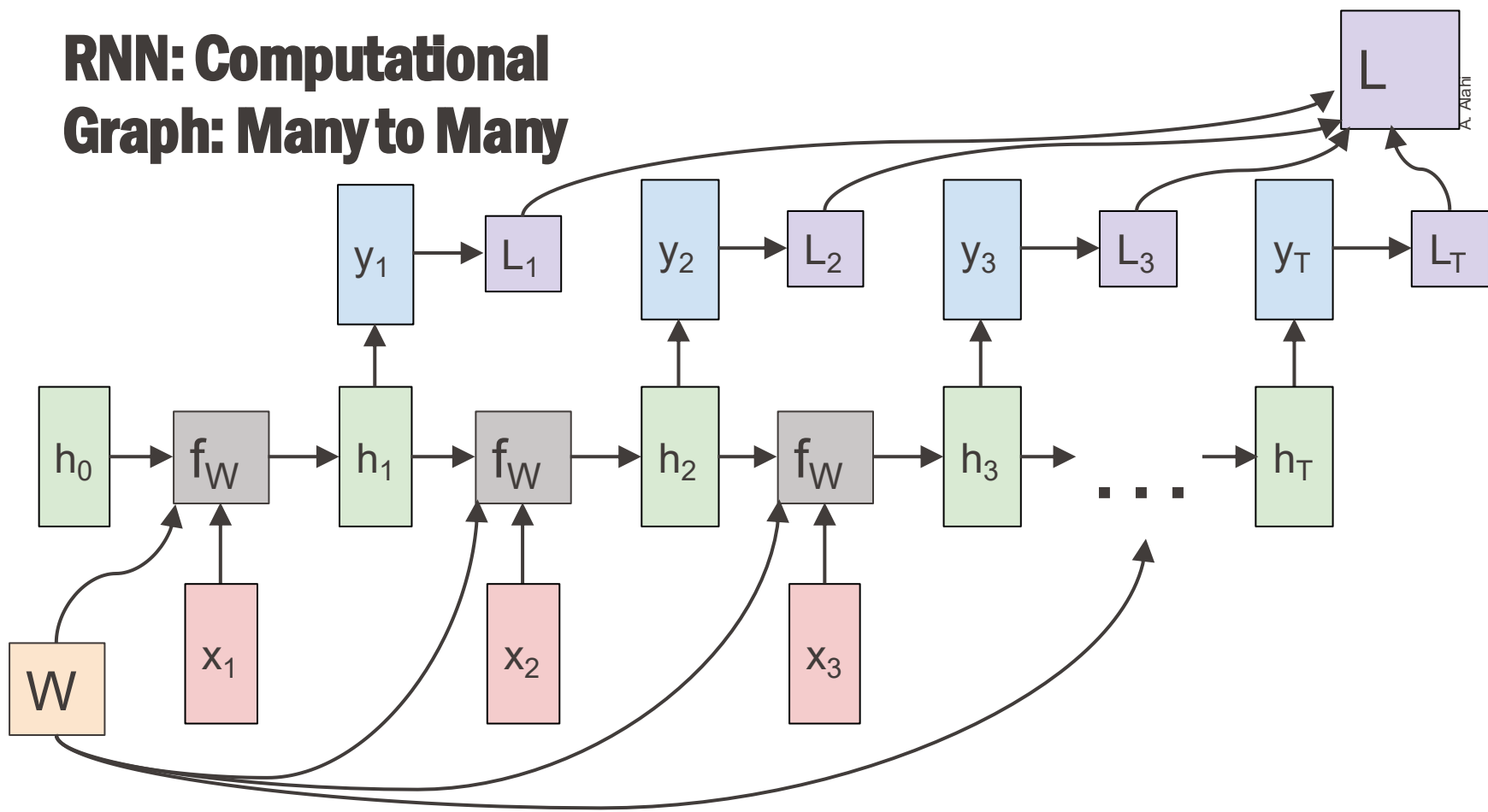
RNN: Computational Graph: Many to Many



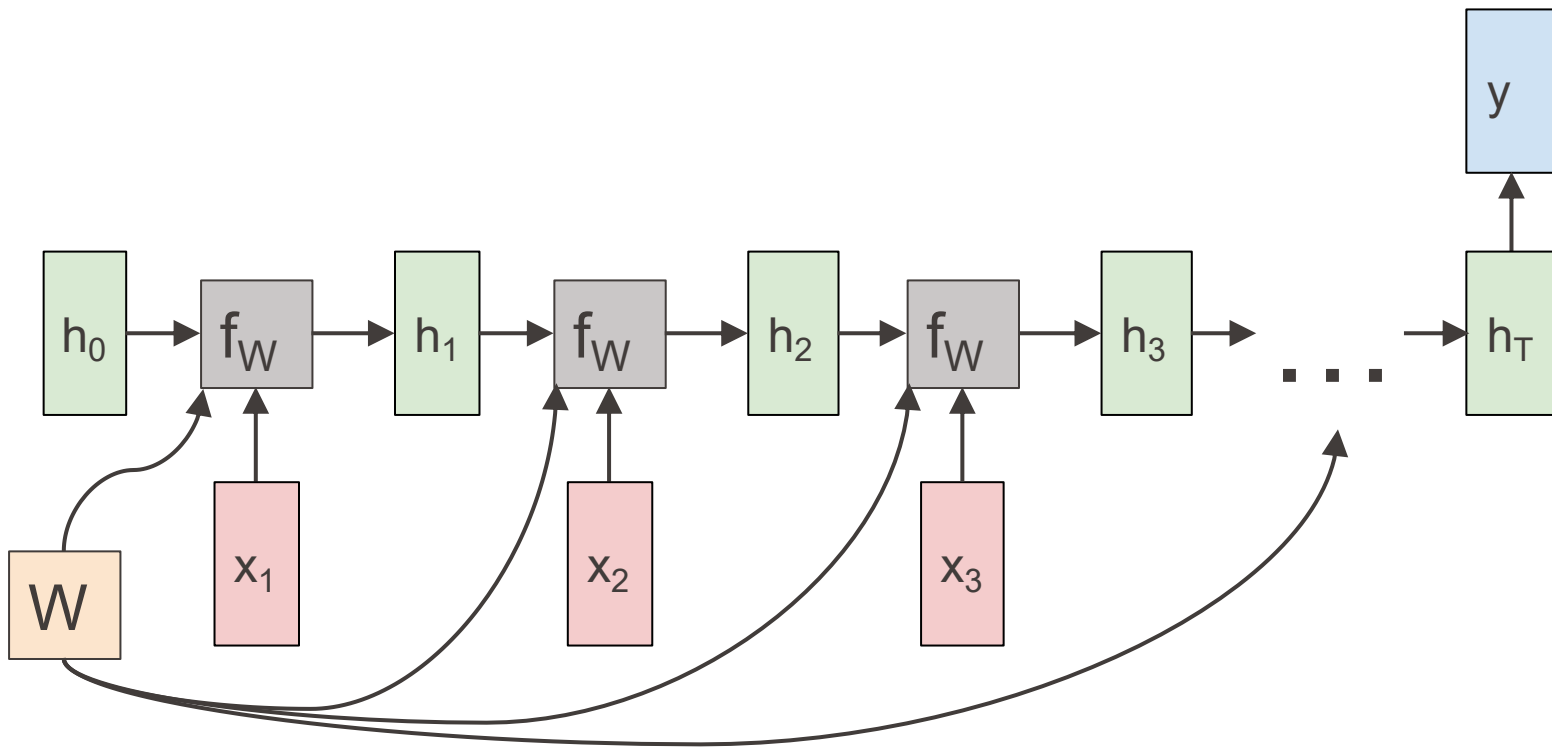
RNN: Computational Graph: Many to Many



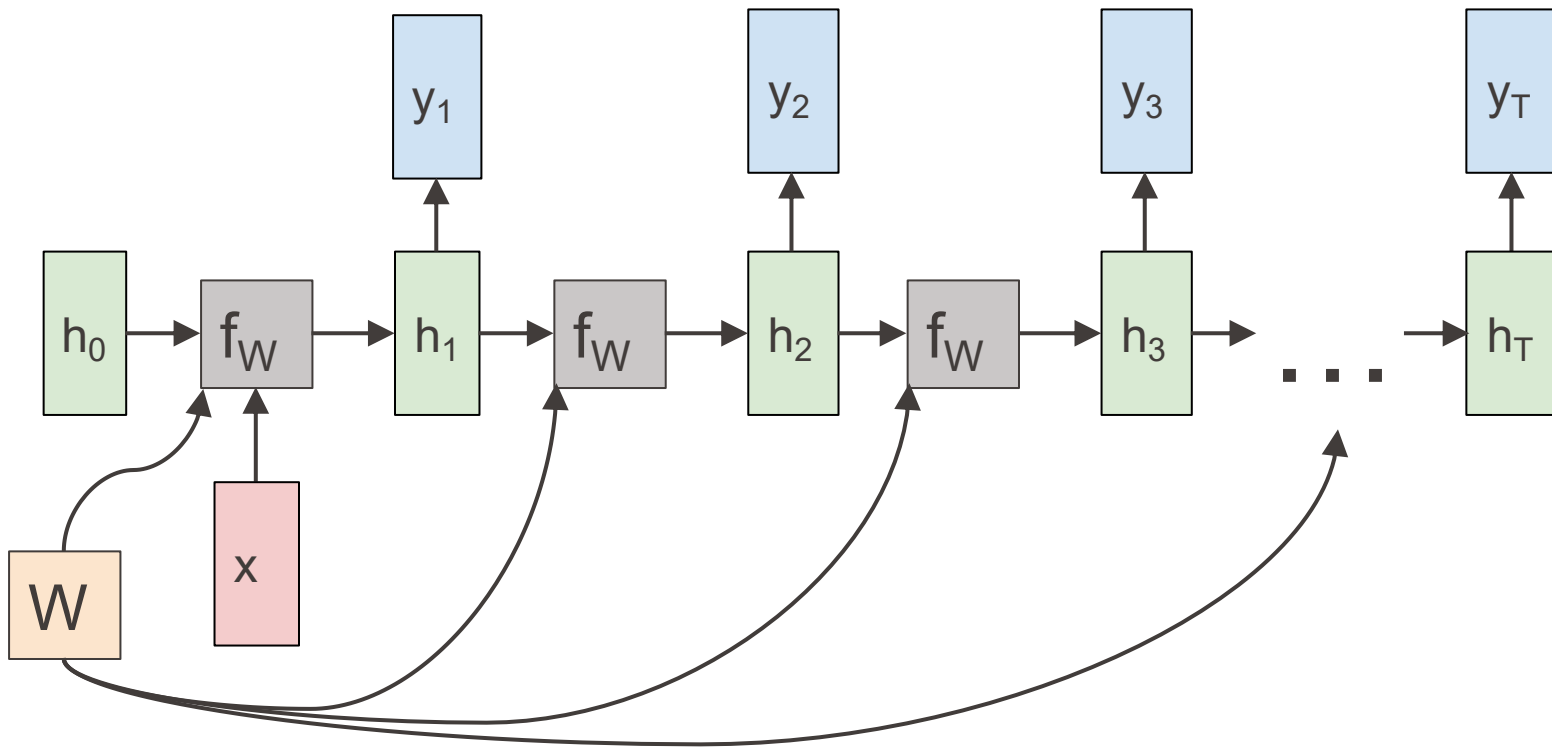
RNN: Computational Graph: Many to Many



RNN: Computational Graph: Many to One

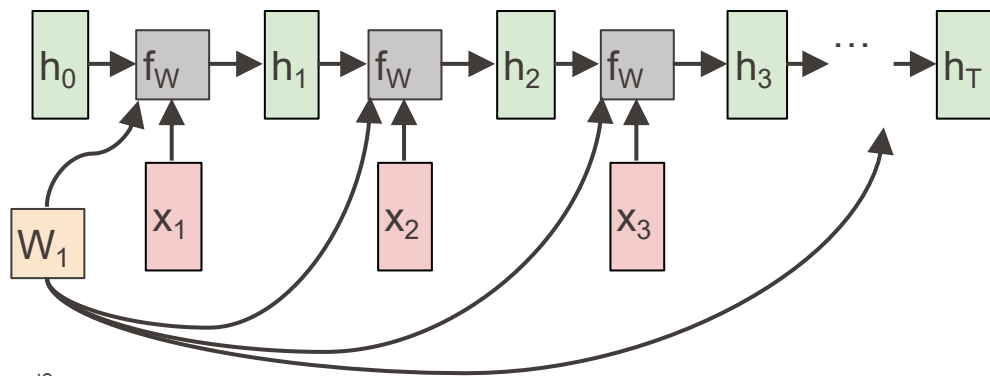


RNN: Computational Graph: One to Many



Sequence to Sequence: Many-to-one + one-to-many

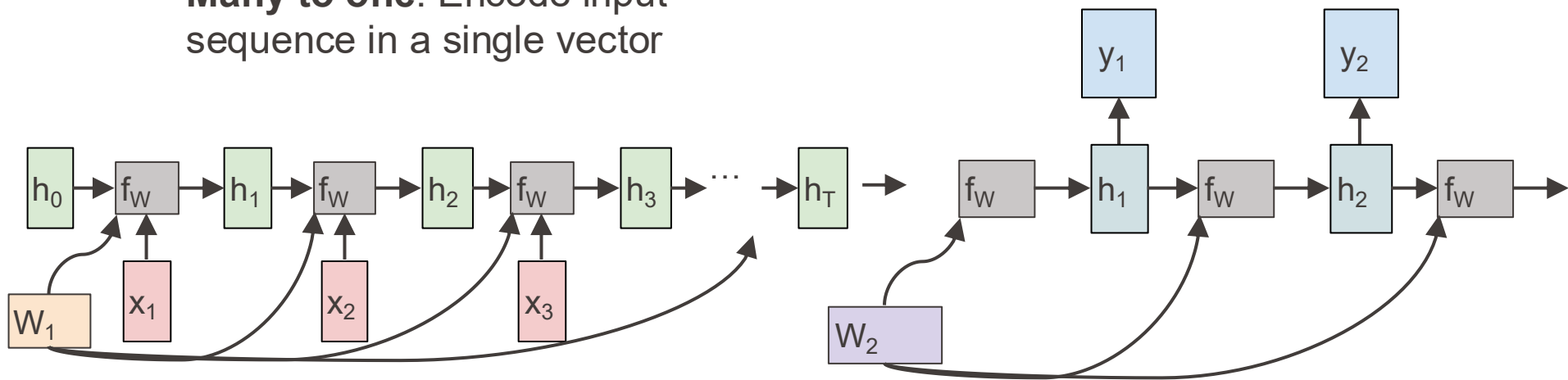
Many to one: Encode input sequence in a single vector



Sequence to Sequence: Many-to-one + one-to-many

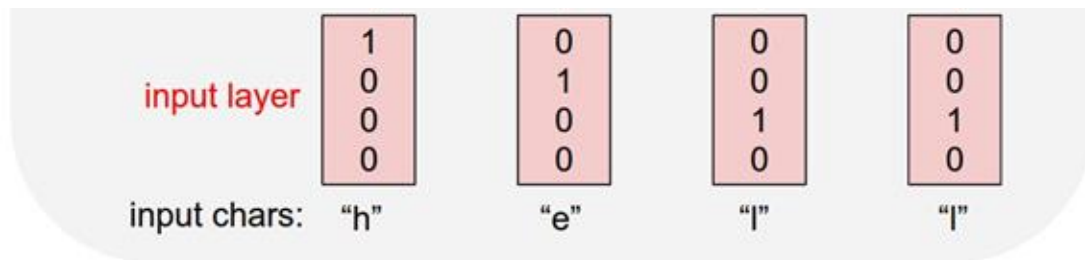
Many to one: Encode input sequence in a single vector

One to many: Produce output sequence from single input vector



Example: Character-level Language Model

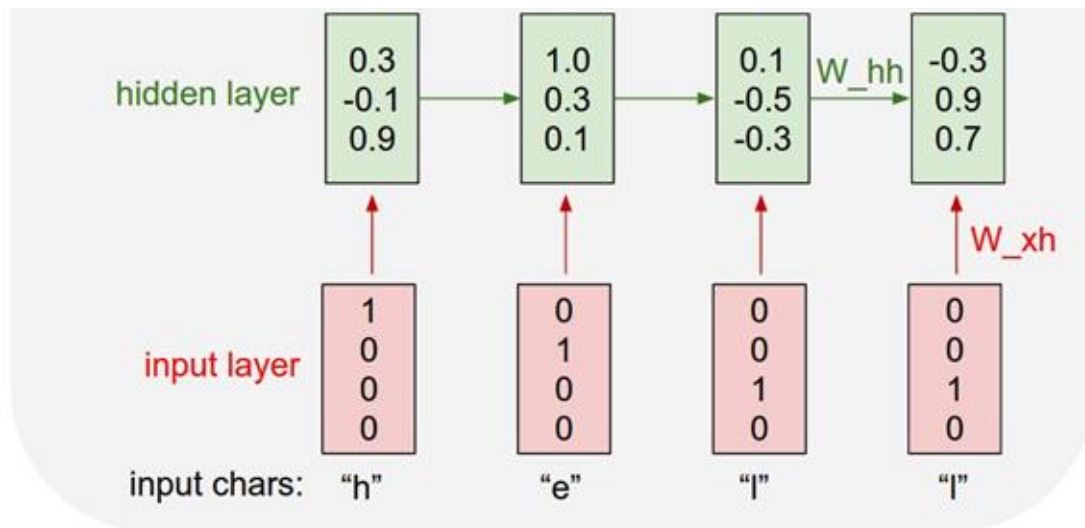
- Vocabulary:
- [h,e,l,o]
- Example training sequence:
- “hello”



Example: Character-level Language Model

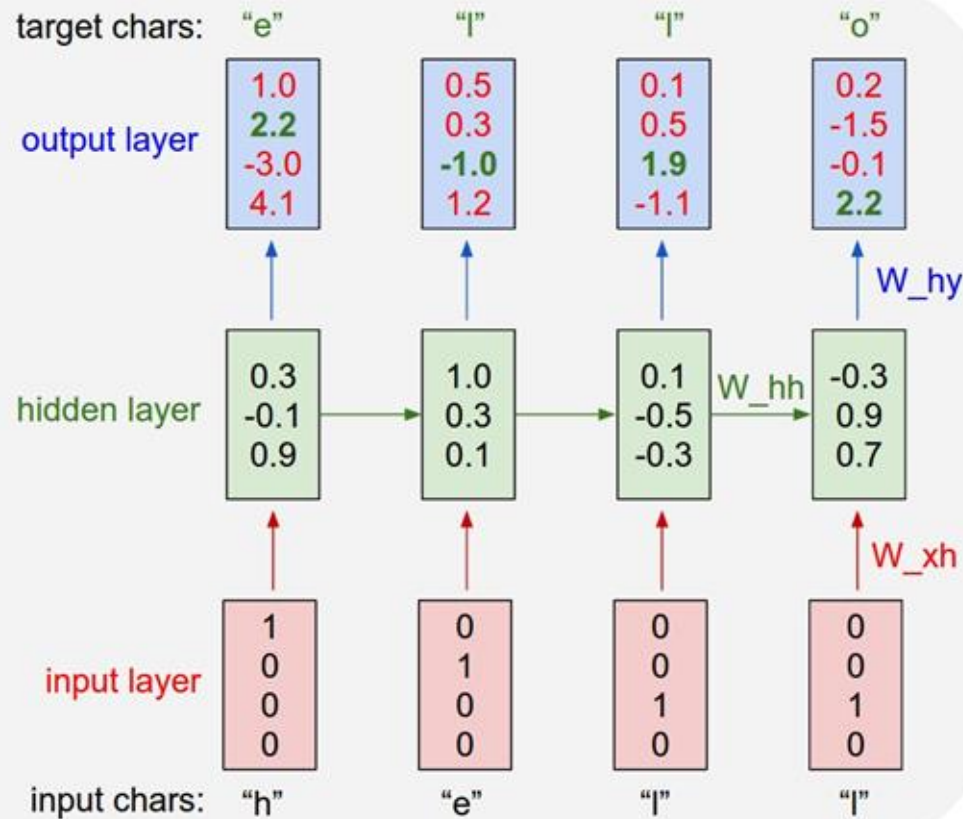
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

- Vocabulary:
- [h,e,l,o]
- Example training
- sequence:
- “hello”



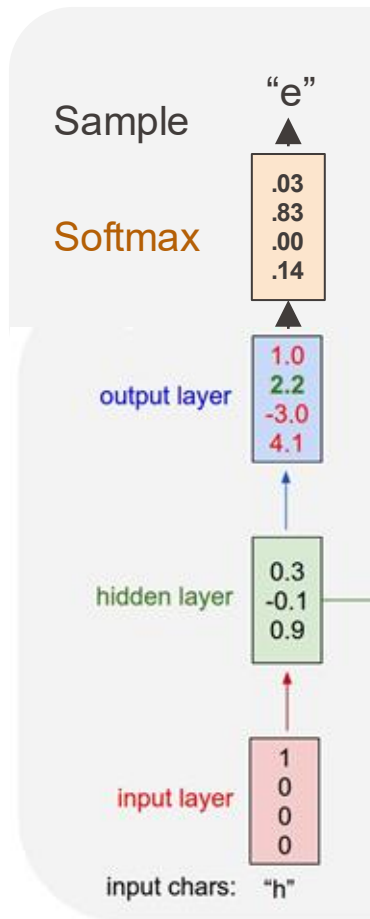
Example: Character-level Language Model

- Vocabulary:
- [h,e,l,o]
- Example training
- sequence:
- “hello”



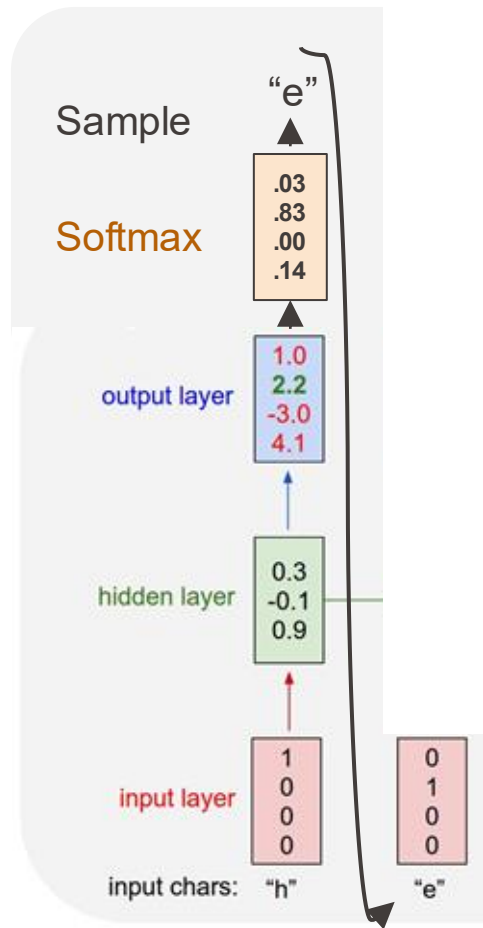
Example: Character-level Language Model Sampling

- Vocabulary:
- [h,e,l,o]
- At test-time sample characters one at a time, feed back to model



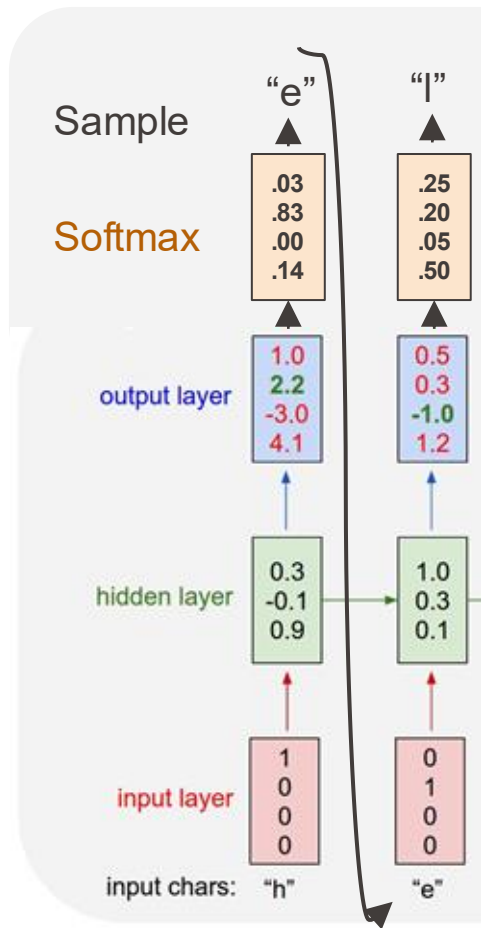
Example: Character-level Language Model Sampling

- Vocabulary:
- [h,e,l,o]
- At test-time sample characters one at a time, feed back to model



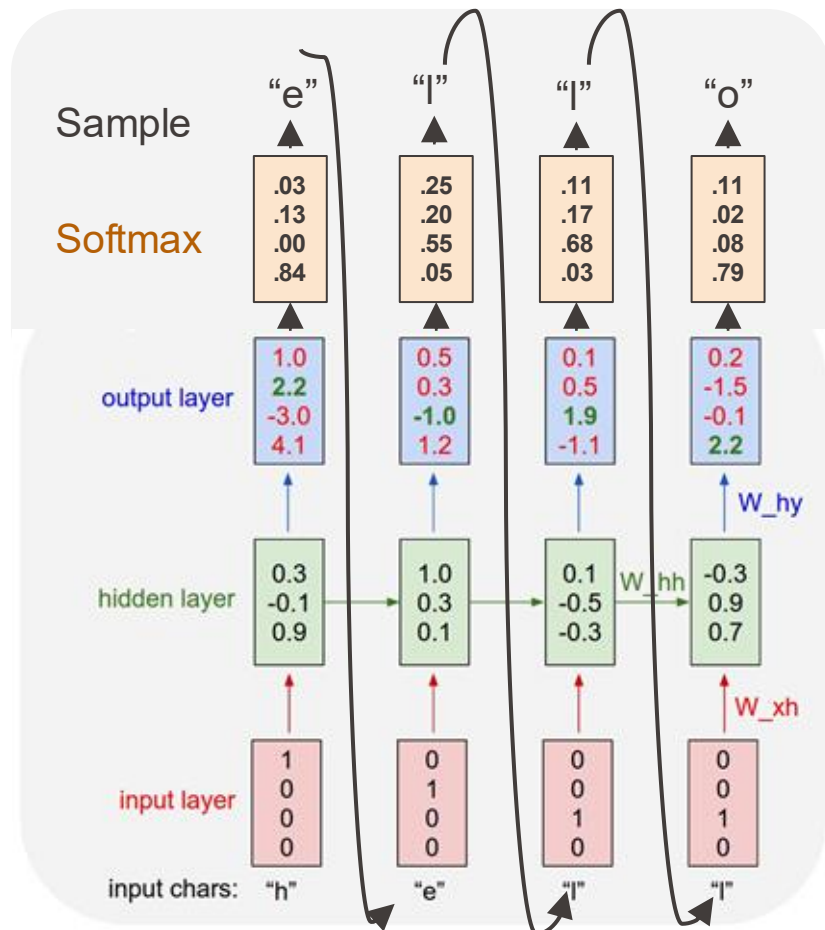
Example: Character-level Language Model Sampling

- Vocabulary:
- [h,e,l,o]
- At test-time sample characters one at a time, feed back to model

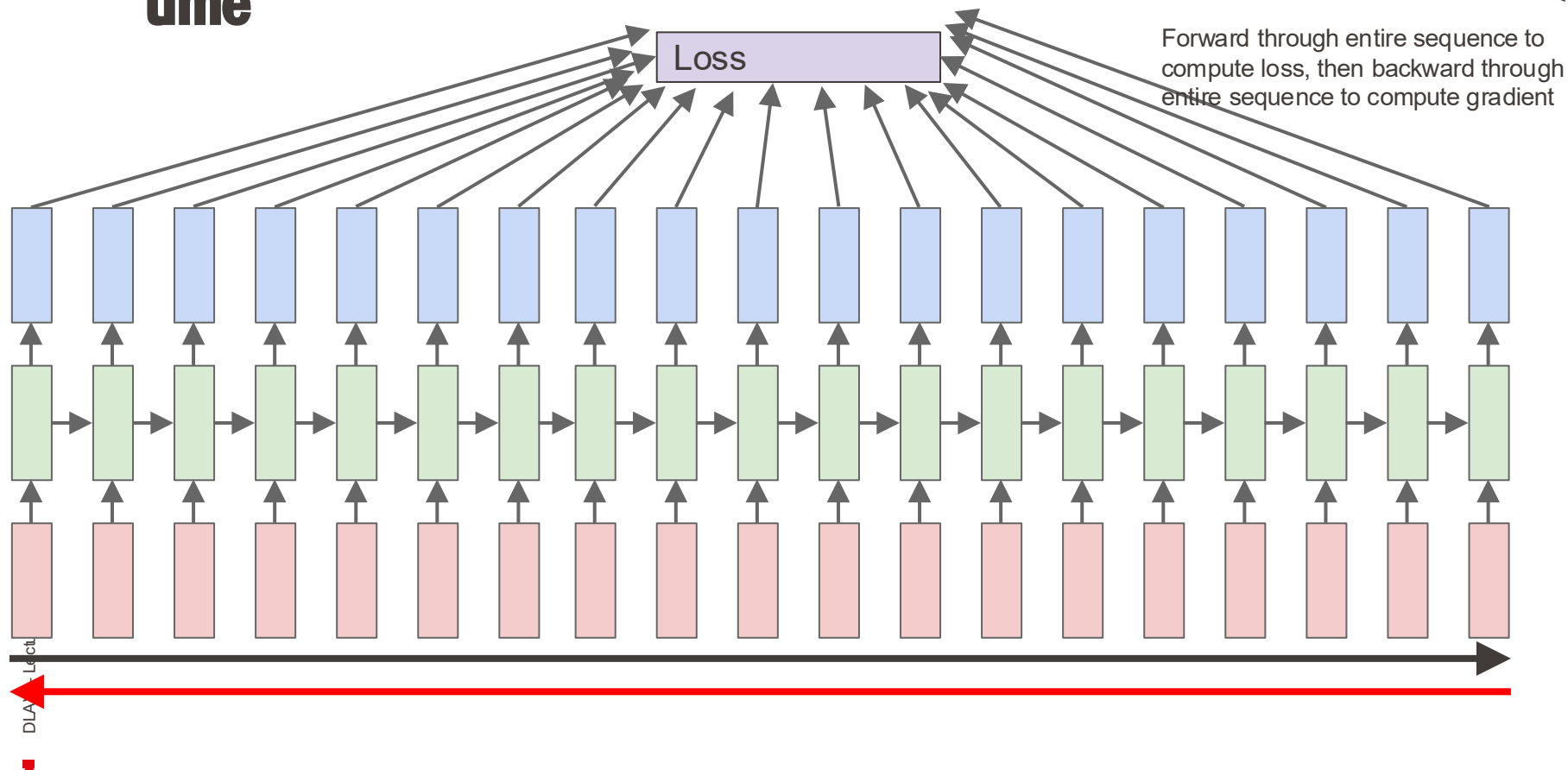


Example: Character-level Language Model Sampling

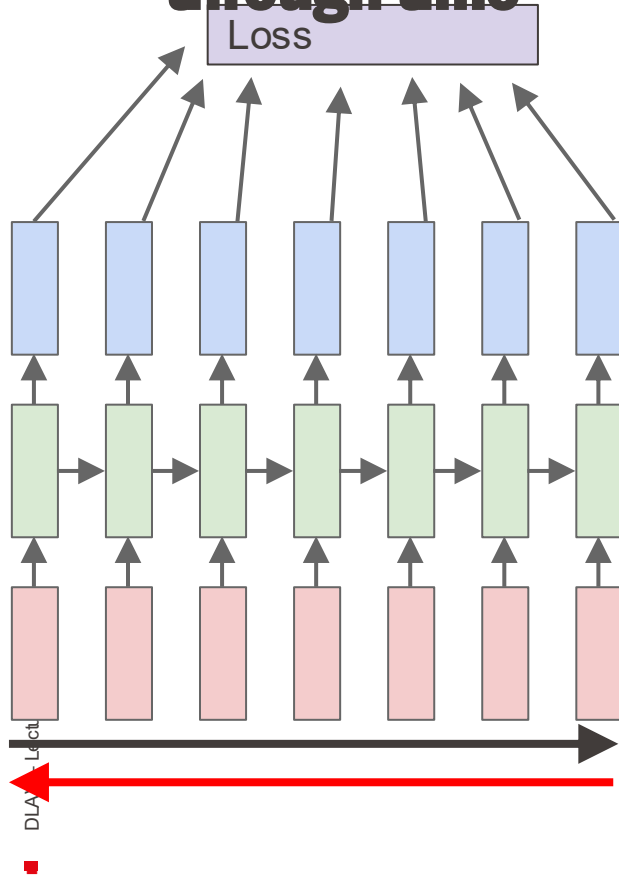
- Vocabulary:
- [h,e,l,o]
- At test-time sample characters one at a time, feed back to model



Backpropagation through time

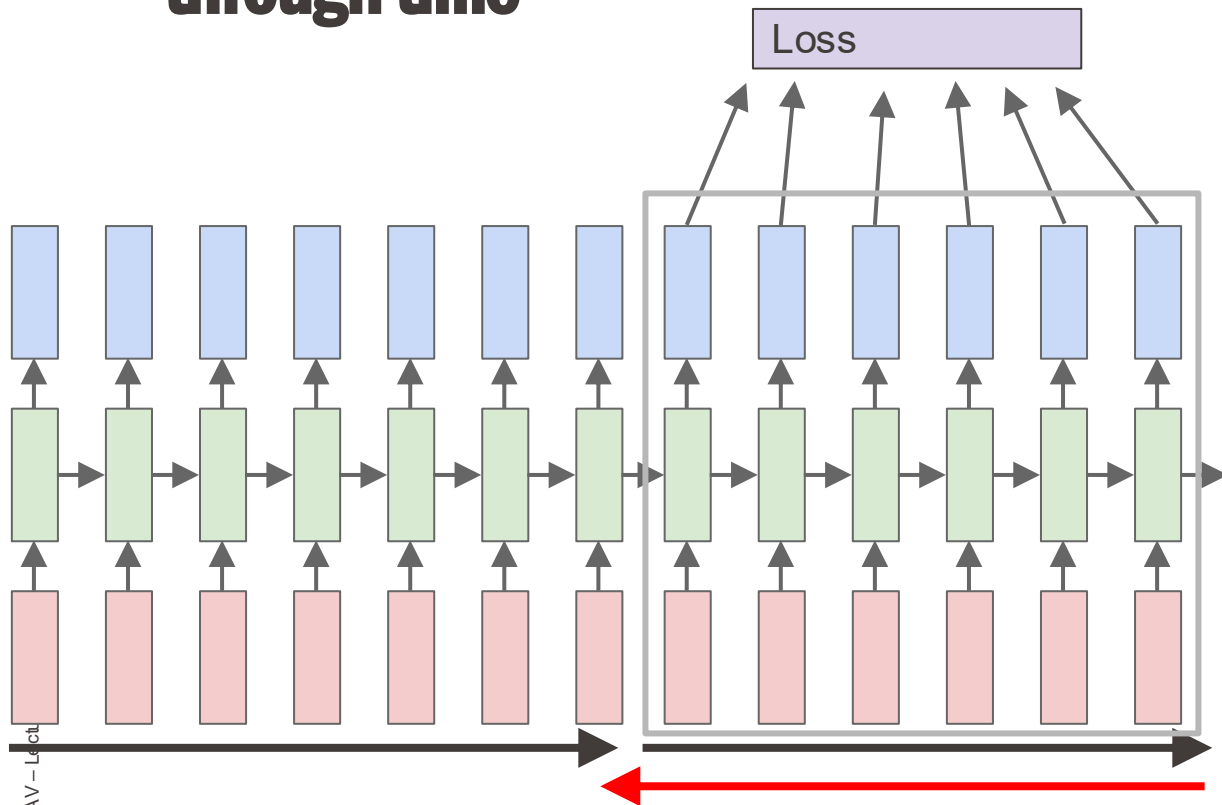


Truncated Backpropagation through time



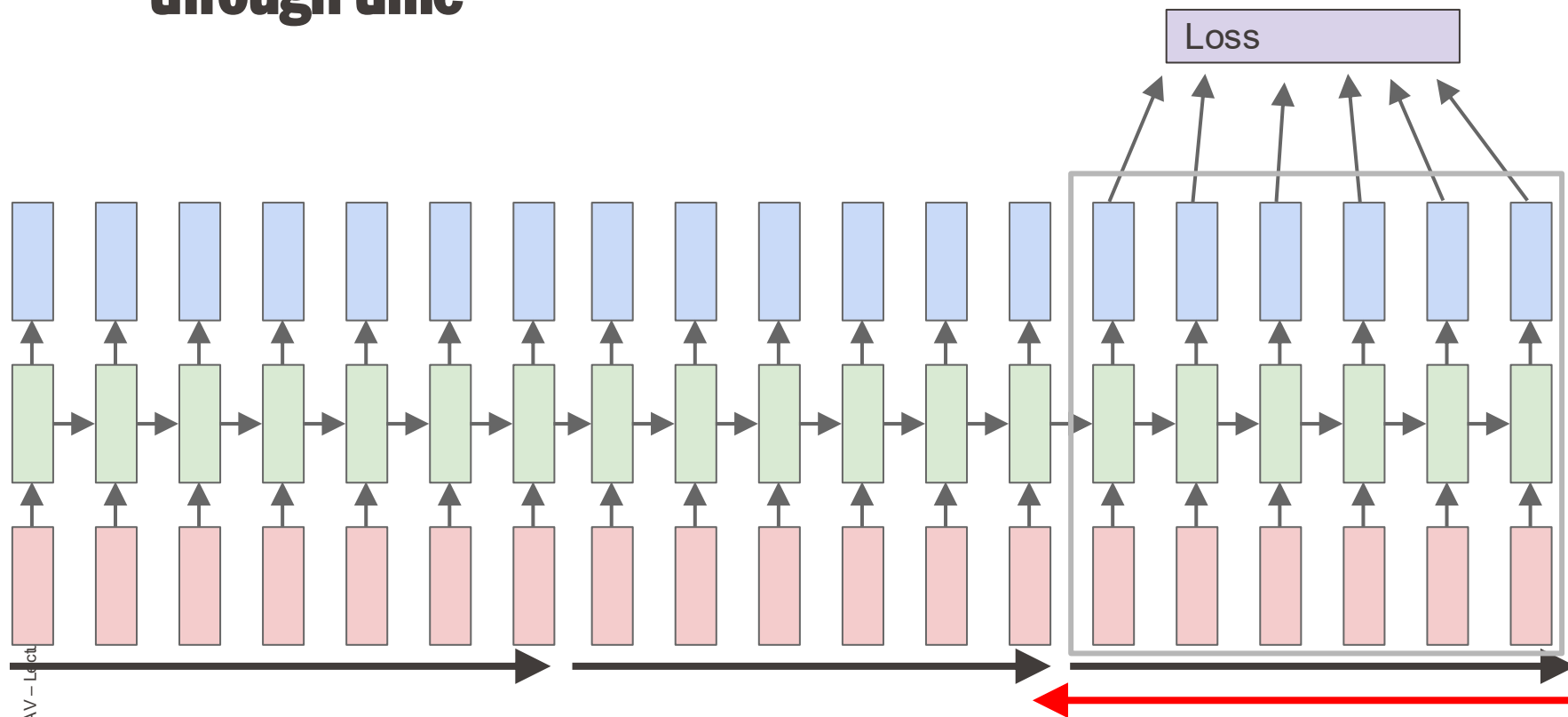
Run forward and backward through chunks of the sequence instead of whole sequence

Truncated Backpropagation through time



Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

Truncated Backpropagation through time



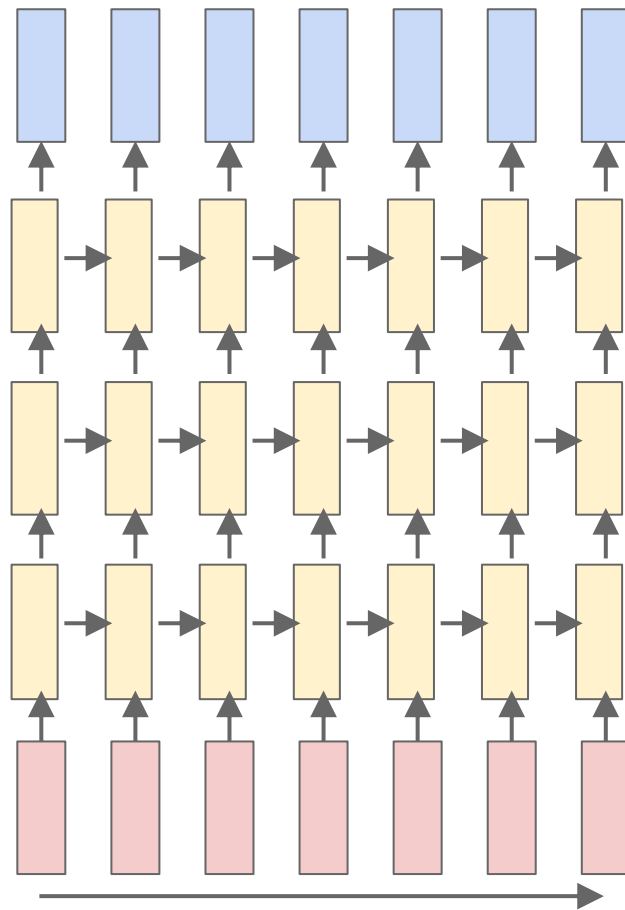
Multilayer RNNs

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

 $h \in \mathbb{R}^n$
 $W^l [n \times 2n]$

depth

time



Multilayer RNNs

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$h \in \mathbb{R}^n, \quad W^l [n \times 2n]$$

LSTM:

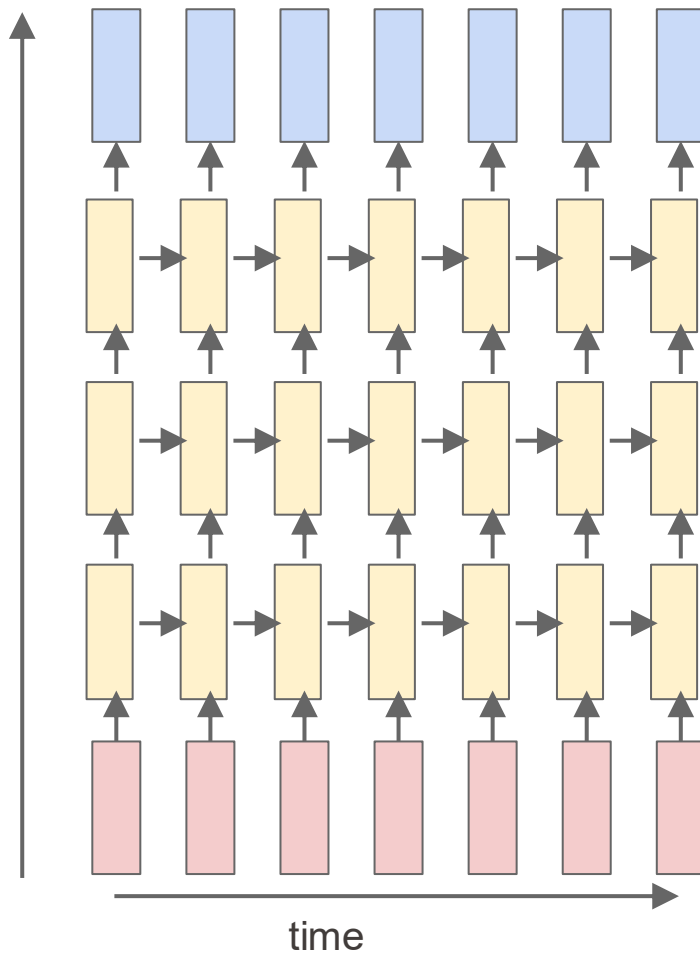
$$W^l [4n \times 2n]$$

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \tanh \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

depth



Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

GRU [*Learning phrase representations using rnn encoder-decoder for statistical machine translation*, Cho et al. 2014]

$$\begin{aligned}
 r_t &= \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\
 z_t &= \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\
 \tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\
 h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t
 \end{aligned}$$

[*LSTM: A Search Space Odyssey*, Greff et al., 2015]

[*An Empirical Exploration of Recurrent Network Architectures*, Jozefowicz et al., 2015]

MUT1:

$$\begin{aligned}
 z &= \text{sigm}(W_{xz}x_t + b_z) \\
 r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\
 h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + \tanh(x_t) + b_h) \odot z \\
 &+ h_t \odot (1 - z)
 \end{aligned}$$

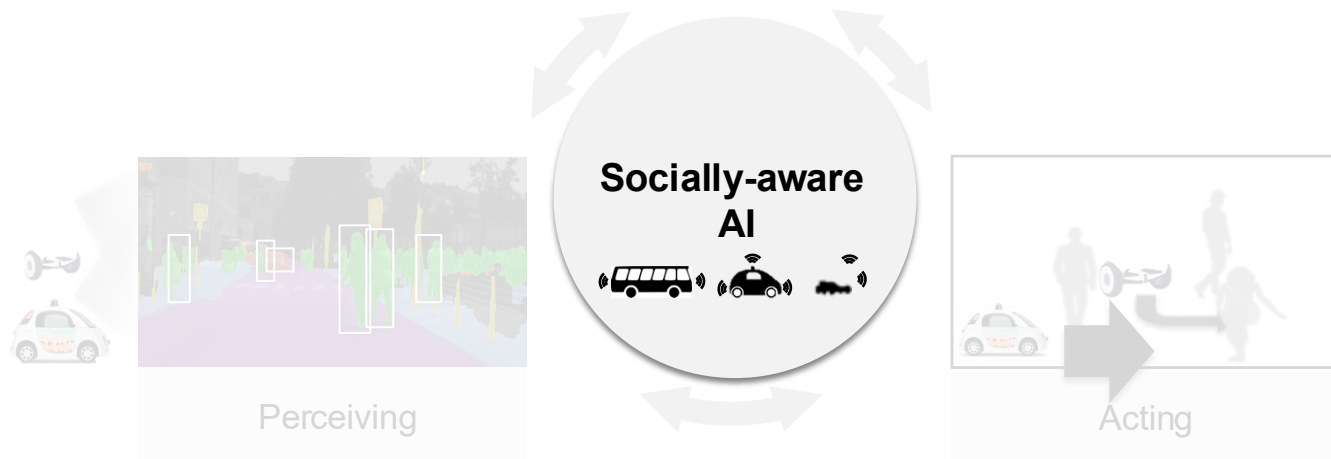
MUT2:

$$\begin{aligned}
 z &= \text{sigm}(W_{xz}x_t + W_{hz}h_t + b_z) \\
 r &= \text{sigm}(x_t + W_{hr}h_t + b_r) \\
 h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\
 &+ h_t \odot (1 - z)
 \end{aligned}$$

MUT3:

$$\begin{aligned}
 z &= \text{sigm}(W_{xz}x_t + W_{hz} \tanh(h_t) + b_z) \\
 r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\
 h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\
 &+ h_t \odot (1 - z)
 \end{aligned}$$

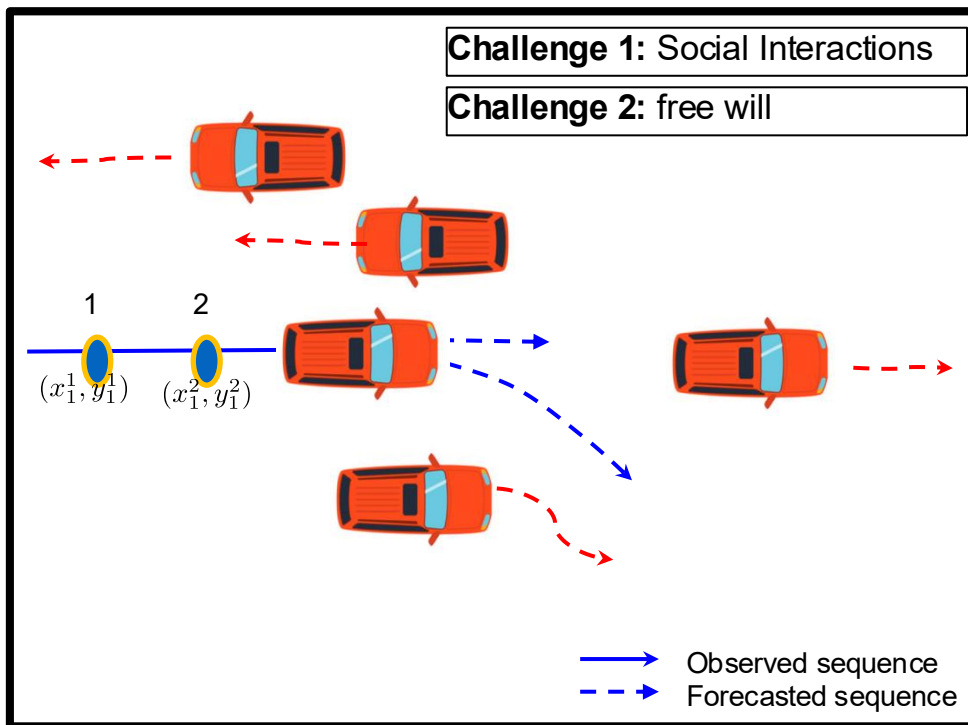
-  RNNs allow a lot of flexibility in architecture design
-  Vanilla RNNs are simple but don't work very well
-  Common to use LSTM or GRU: their additive interactions improve gradient flow
-  Backward flow of gradients in RNN can explode or vanish. Exploding is controlled with gradient clipping. Vanishing is controlled with additive interactions (LSTM)
-  Better/simpler architectures are a hot topic of current research
-  Better understanding (both theoretical and empirical) is needed.



Social Forecasting

Input: Given a sequence of states, e.g., (x^t, y^t) coordinates in time

Output: **Predict** the future states, e.g., next 5 seconds



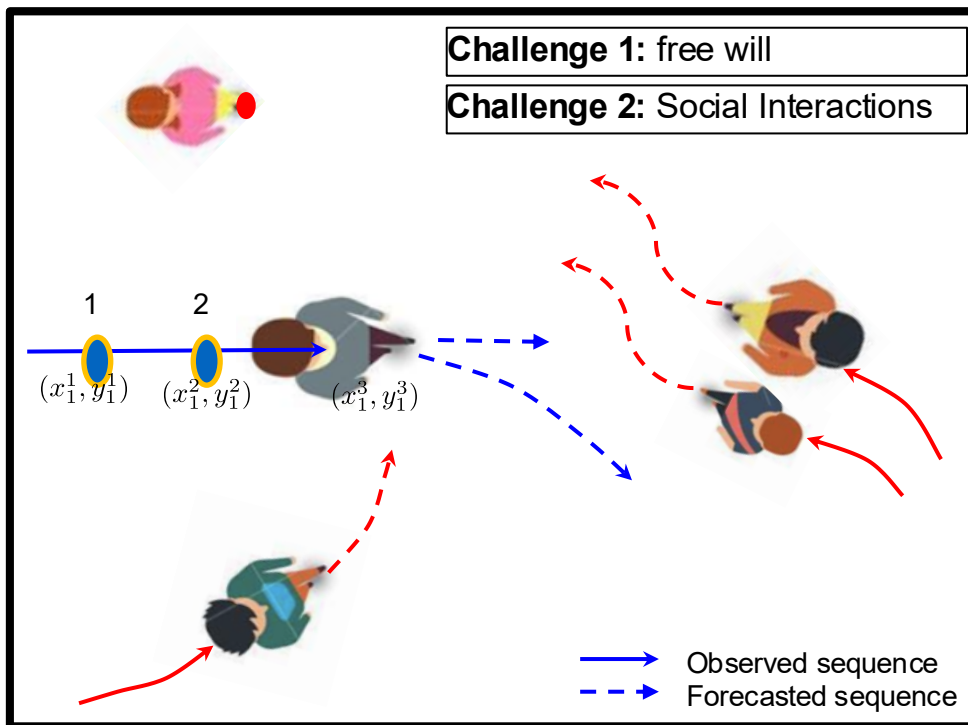
Forecasting cars



Social Forecasting

Input: Given a sequence of states, e.g., (x^t, y^t) coordinates in time

Output: **Predict** the future states, e.g., next 5 seconds

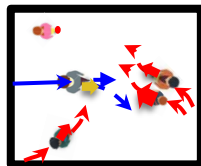


Social Forecasting

- Previous works

Knowledge-driven

- Social Forces Model [1],
 $F = F^{\text{attractive}} + F^{\text{repulsive}} \dots$



- Discrete Choice Model [2]

$$\underbrace{U}_{\text{Utility}} = \underbrace{V}_{\text{Systematic}} + \underbrace{\varepsilon}_{\text{Random}}$$

- ✓ Interpretability
- X Predictability

[1] Helbing *et al.*, Physical review, '95

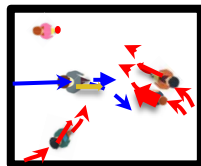
[2] Antonini *et al.*, Transportation Research, '06

Social Forecasting

Previous works

Knowledge-driven

- Social Forces Model [1],
 $F = F_{\text{attractive}} + F_{\text{repulsive}} \dots$



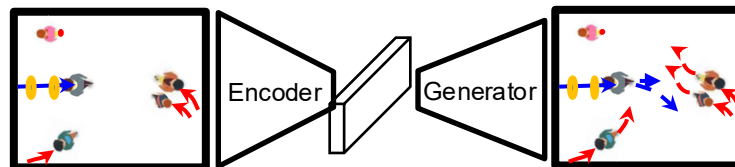
- Discrete Choice Model [2]

$$\underbrace{U}_{\text{Utility}} = \underbrace{V}_{\text{Systematic}} + \underbrace{\varepsilon}_{\text{Random}}$$

- ✓ Interpretability
- X Predictability

Our works

Data-driven methods



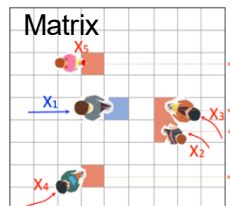
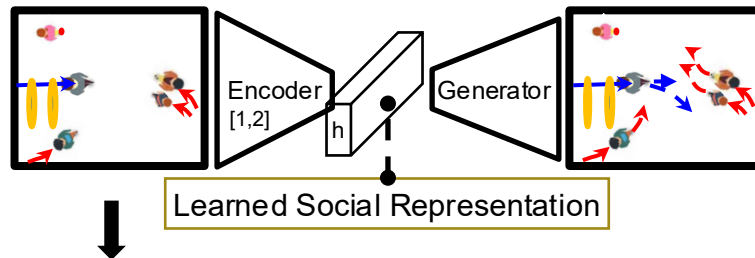
[1] Helbing *et al.*, Physical review, '95

[2] Antonini *et al.*, Transportation Research, '06

Social Forecasting

Learning Representation of Crowds

Challenge 1: Social Interactions



$$H_i^t(m, n, :) = \sum_{j \in N_i} \mathbb{1}_{mn} [x_j^t - x_i^t, y_j^t - y_i^t]$$

$$h^t = \text{LSTM}(H^t, h^{t-1}, W_1)$$

$$(x^{t+1}, y^{t+1}) = f(h^t, W_2)$$

Where W_1, W_2 are parameters to be learned,
 f is a non linear neural network function

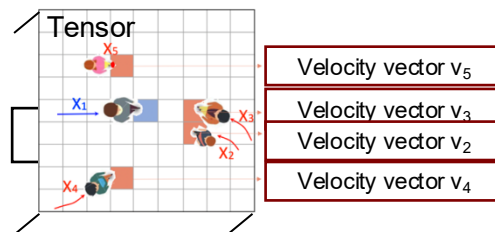
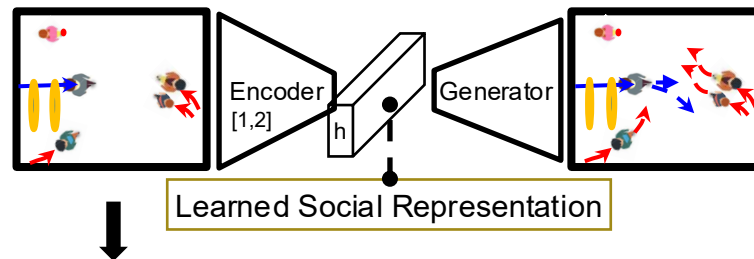
[1] Social LSTM (Long Short-Term Memory), CVPR'16

[2] Human trajectory forecasting: a deep learning perspective, IEEE Transactions on ITS'21

Social Forecasting

Learning Representation of Crowds

Challenge 1: Social Interactions



$$H_i^t(m, n, :) = \sum_{j \in N_i} \mathbb{1}_{mn} [x_j^t - x_i^t, y_j^t - y_i^t] \quad v_t$$

$$h^t = \text{LSTM}(H^t, h^{t-1}, W_1)$$

$$(x^{t+1}, y^{t+1}) = f(h^t, W_2)$$

Where W_1, W_2 are parameters to be learned,
 f is a non linear neural network function

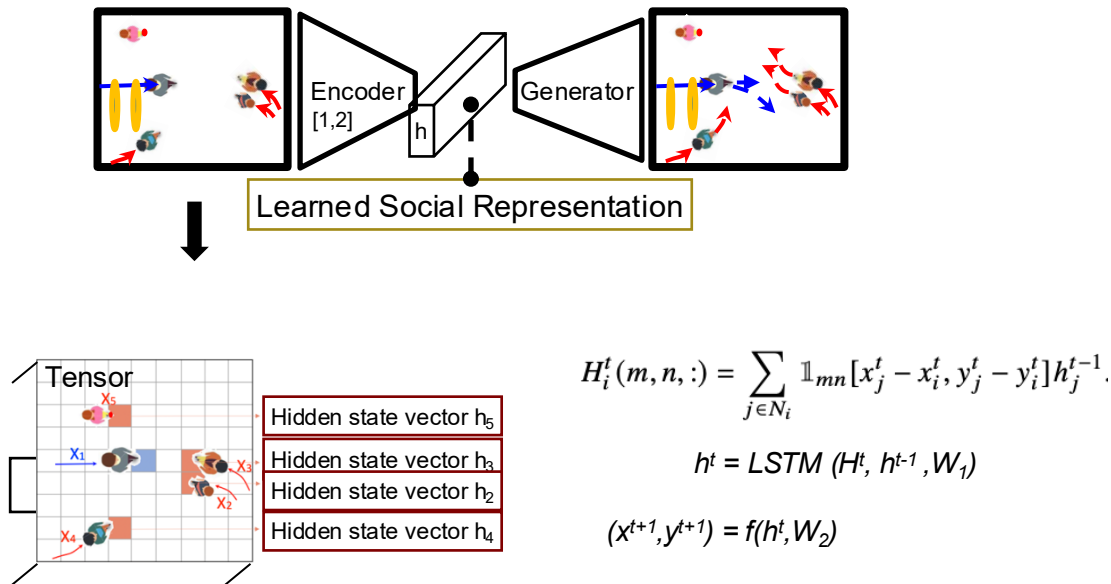
[1] Social LSTM (Long Short-Term Memory), CVPR'16

[2] Human trajectory forecasting: a deep learning perspective, IEEE Transactions on ITS'21

Social Forecasting

Learning Representation of Crowds

Challenge 1: Social Interactions



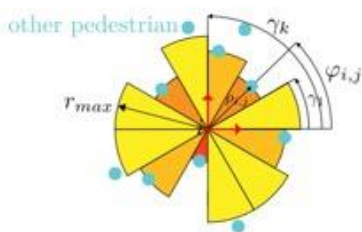
Where W_1, W_2 are parameters to be learned,
 f is a non linear neural network function

[1] Social LSTM (Long Short-Term Memory), CVPR'16

[2] Human trajectory forecasting: a deep learning perspective, IEEE Transactions on ITS'21

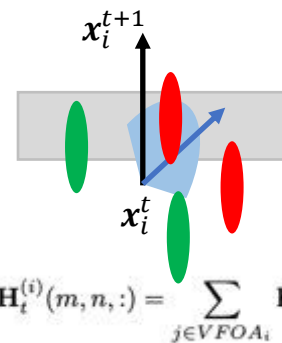
Modified Grid-based Designs

Radial Grids [1]



Value encoded in each angular grid cell is minimum distance to neighbors within the cone

View frustum of attention (VOFA) [2]



$$\mathbf{H}_t^{(i)}(m, n, :) = \sum_{j \in \text{VFOA}_i} \mathbf{h}_t^{(j)}$$

Only consider neighbors within the VOFA (red). The axis of VOFA is determined by the head pose.

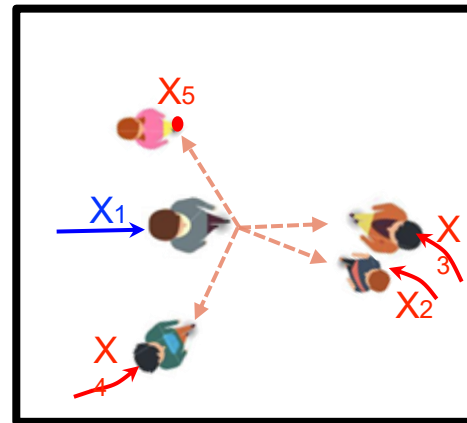
- [1] Pfeiffer, Mark et al. "A Data-driven Model for Interaction-Aware Pedestrian Motion Prediction in Object Cluttered Environments." *ICRA*, 2018.
 [2] Hasan, Irtiza et al. "MX-LSTM: Mixing Tracklets and Vislets to Jointly Forecast Trajectories and Head Poses." *CVPR* 2018

Non-Grid based Modules: Input States

Design Choice of these modules is driven by:

A. Neighbor Input State

1. Position (~~Absolute~~ / *Relative*)
2. Velocity [1]
3. Hidden-state [2, 3]
4. Others: e.g., Distance of closest approach[4]



[1] B. Ivanovic and M. Pavone, "The trajectron: Probabilistic multi-agent trajectory modeling with dynamic spatiotemporal graphs," ICCV 2019.

[2] A. Gupta, et al. "Social gan: Socially acceptable trajectories with generative adversarial networks," CVPR 2018.

[3] V. Kosaraju et al., "Social-bigat: Multimodal trajectory forecasting using bicycle-gan and graph attention networks," NeurIPS, 2019.

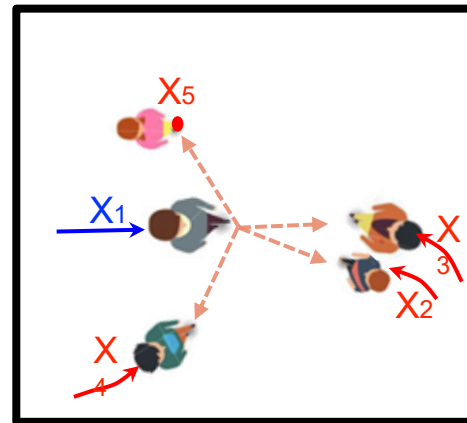
[4] Amirian, Javad et al. "Social Ways: Learning Multi-Modal Distributions of Pedestrian Trajectories With GANs." *CVPR Workshop, 2019*

Non-Grid based Modules: State Embedding

Design Choice of these modules is driven by:

B. State embedding module

1. MLP [1, 2]
2. LSTM (monitors how relative state evolves over time) [3, 4]



- [1] B. Ivanovic and M. Pavone, "The trajetron: Probabilistic multi-agent trajectory modeling with dynamic spatiotemporal graphs," ICCV 2019.
- [2] A. Gupta, et al. "Social gan: Socially acceptable trajectories with generative adversarial networks," CVPR 2018.
- [3] V. Kosaraju et al., "Social-bigat: Multimodal trajectory forecasting using bicycle-gan and graph attention networks," NeurIPS, 2019.
- [4] Amirian, Javad et al. "Social Ways: Learning Multi-Modal Distributions of Pedestrian Trajectories With GANs." *CVPR Workshop, 2019*

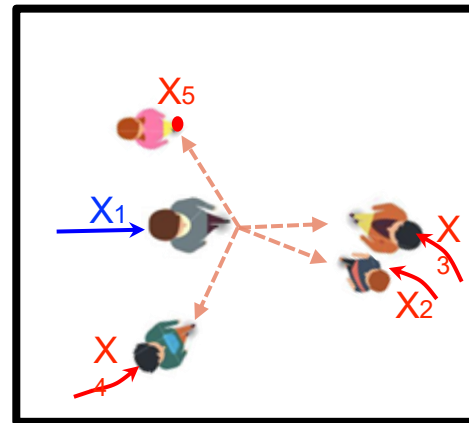
Non-Grid based Modules: State Embedding

Design Choice of these modules is driven by:

B. State embedding module

1. MLP [1, 2]
2. LSTM (monitors how relative state evolves over time) [3, 4]

C. Attention?

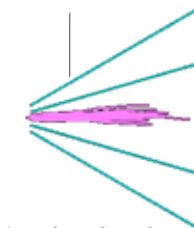
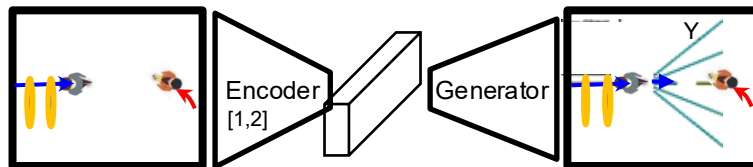


- [1] B. Ivanovic and M. Pavone, "The trajetron: Probabilistic multi-agent trajectory modeling with dynamic spatiotemporal graphs," ICCV 2019.
- [2] A. Gupta, et al. "Social gan: Socially acceptable trajectories with generative adversarial networks," CVPR 2018.
- [3] V. Kosaraju et al., "Social-bigat: Multimodal trajectory forecasting using bicycle-gan and graph attention networks," NeurIPS, 2019.
- [4] Amirian, Javad et al. "Social Ways: Learning Multi-Modal Distributions of Pedestrian Trajectories With GANs." *CVPR Workshop, 2019*

Social Forecasting

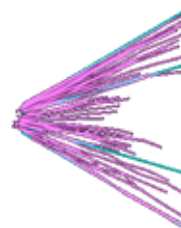
Learning Representation of Crowds

Challenge 2: Multimodality distribution



Average L2
Loss

$$\ell_{Recon} = \frac{1}{K} \sum_k \|Y_i - \hat{Y}_i^{(k)}\|$$



Winner-takes-all
Loss

$$\mathcal{L}_{variety} = \min_k \|Y_i - \hat{Y}_i^{(k)}\|$$

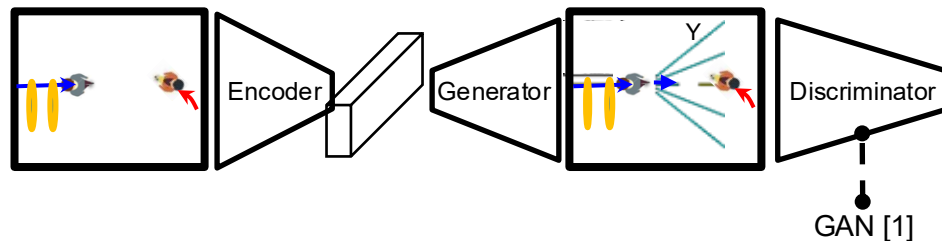
[1] Social LSTM (Long Short-Term Memory), CVPR'16

[2] Human trajectory forecasting: a deep learning perspective, IEEE Transactions on ITS'21

Social Forecasting

Learning Representation of Crowds

Challenge 2: Multimodality distribution



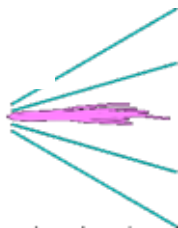
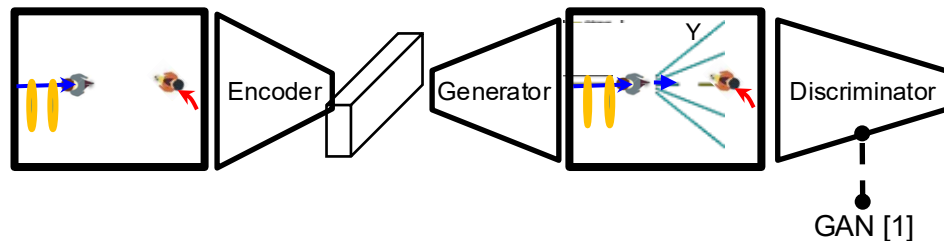
$$\min_G \max_D \mathbb{E}_{x \sim p_r} [\log(D(x))] + \mathbb{E}_{z \sim p_z} [1 - \log(D(G(z)))].$$

G = Generator
D = Discriminator
x = Real sample
 p_r = Real distribution
z = Latent variable
 p_z = Latent distribution

Social Forecasting

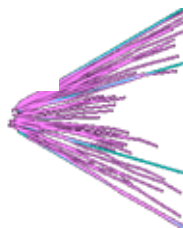
Learning Representation of Crowds

Challenge 2: Multimodality distribution



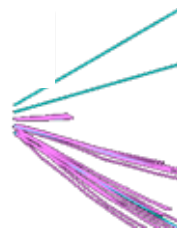
Average L2
Loss

$$\ell_{Recon} = \frac{1}{K} \sum_k \|Y_i - \hat{Y}_i^{(k)}\|$$



Winner-takes-all
Loss

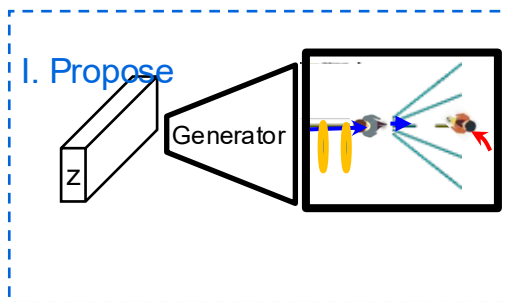
$$\mathcal{L}_{variety} = \min_k \|Y_i - \hat{Y}_i^{(k)}\|$$



Adversarial
Loss [1]

Generative Adversarial Networks [1]

Test time



G = Generator
D = Discriminator
 G_l = layer l of the generator
 x_l = the input to layer l

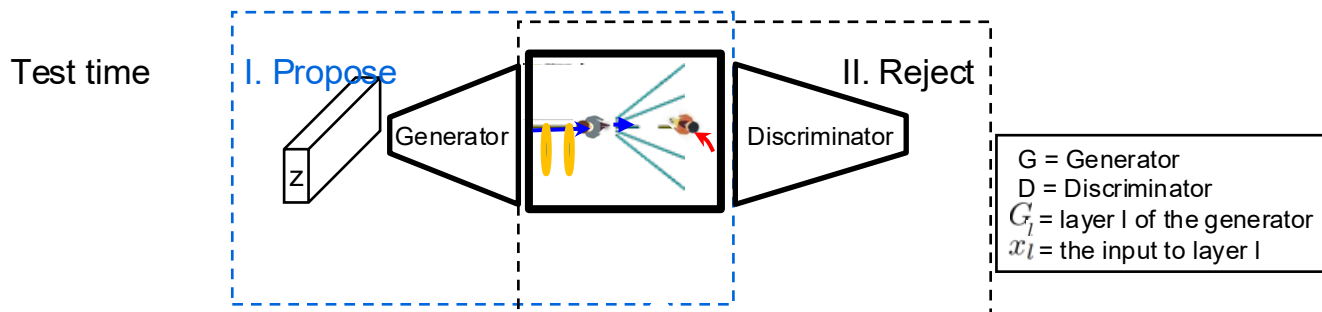
I. Propose:

$$G = G_L \circ G_{L-1} \circ \dots \circ G_1(x_l)$$



Limitation (in practice)
Generator is not optimal

Sampling Strategies in Generative Adversarial Networks [1,2]



I. Propose:

$$G = G_L \circ G_{L-1} \circ \dots \circ G_1(x_i)$$

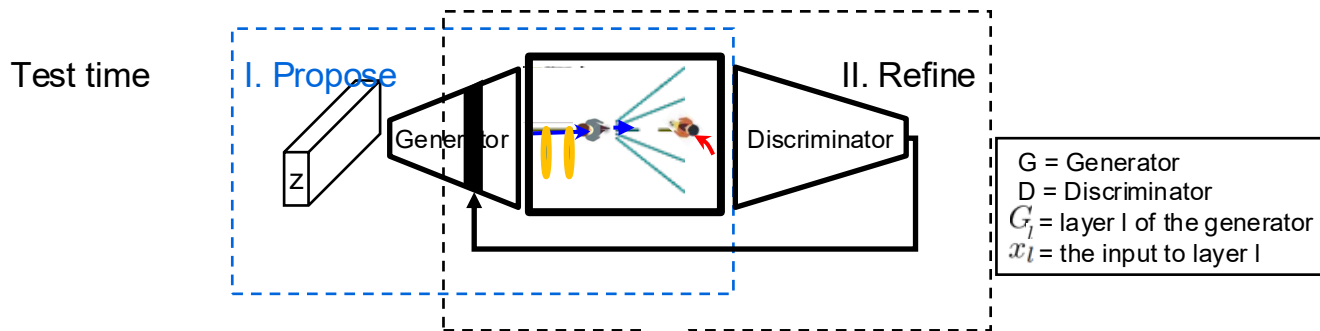
II. Reject:

- Rejection sampling [1]
- Metropolis-Hasting w/ Independence Sampler [2]

[1] S. Azadi et al., Discriminator Rejection Sampling, ICLR'19

[2] R. Turner et al., Metropolis-Hasting GAN, ICML'19

Collaborative Sampling in Generative Adversarial Networks [1]



I. Propose: $G = G_L \circ G_{L-1} \circ \dots \circ G_1(x_i)$

II. Refine: $x_l^{k+1} = x_l^k - \lambda \nabla_l \mathcal{L}_G(x_l^k)$, where $\mathcal{L}_G = -\mathbb{E}_{z \sim p_z} [\log D(G(z))]$

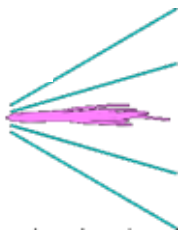
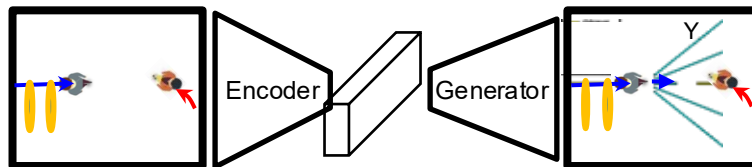
$$x^{k+1} = G_L \circ G_{L-1} \circ \dots \circ G_l(x_l^{k+1}),$$

Until: $D(x^{k+1}) > \eta$

Social Forecasting

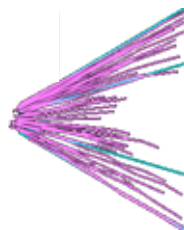
Learning Representation of Crowds

Challenge 2: Multimodality distribution



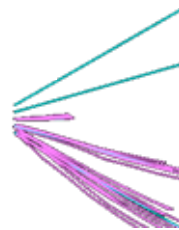
Average L2
Loss

$$\ell_{Recon} = \frac{1}{K} \sum_k \|Y_i - \hat{Y}_i^{(k)}\|$$

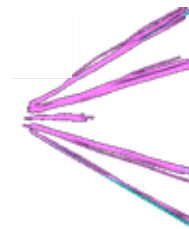


Winner-takes-all
Loss

$$\mathcal{L}_{variety} = \min_k \|Y_i - \hat{Y}_i^{(k)}\|$$



Adversarial
Loss [1]



Adversarial
Loss w/
Collab. Sampling [2]

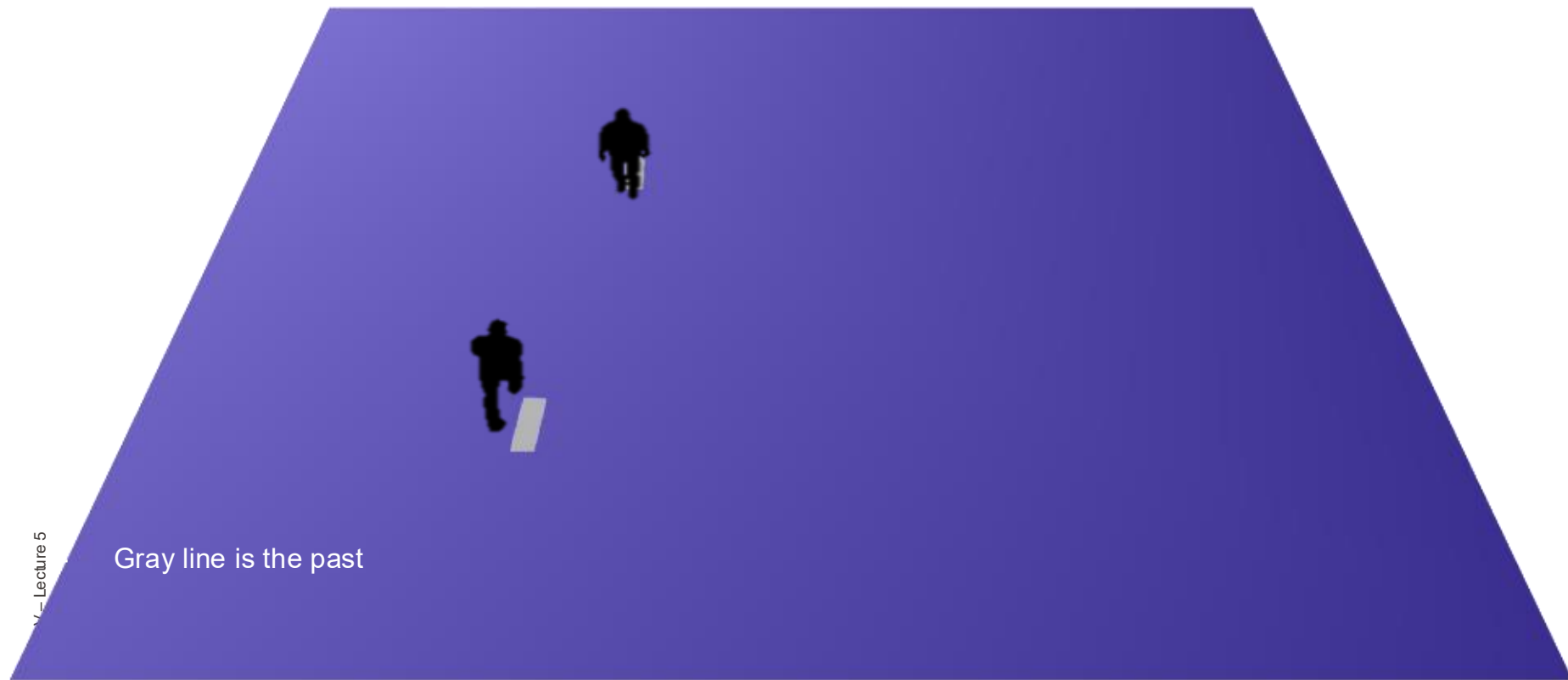
[1] Social GAN (Generative Adversarial Network), CVPR'18

[2] Collaborative Sampling in Generative Adversarial Networks, AAAI'20

Two pedestrians

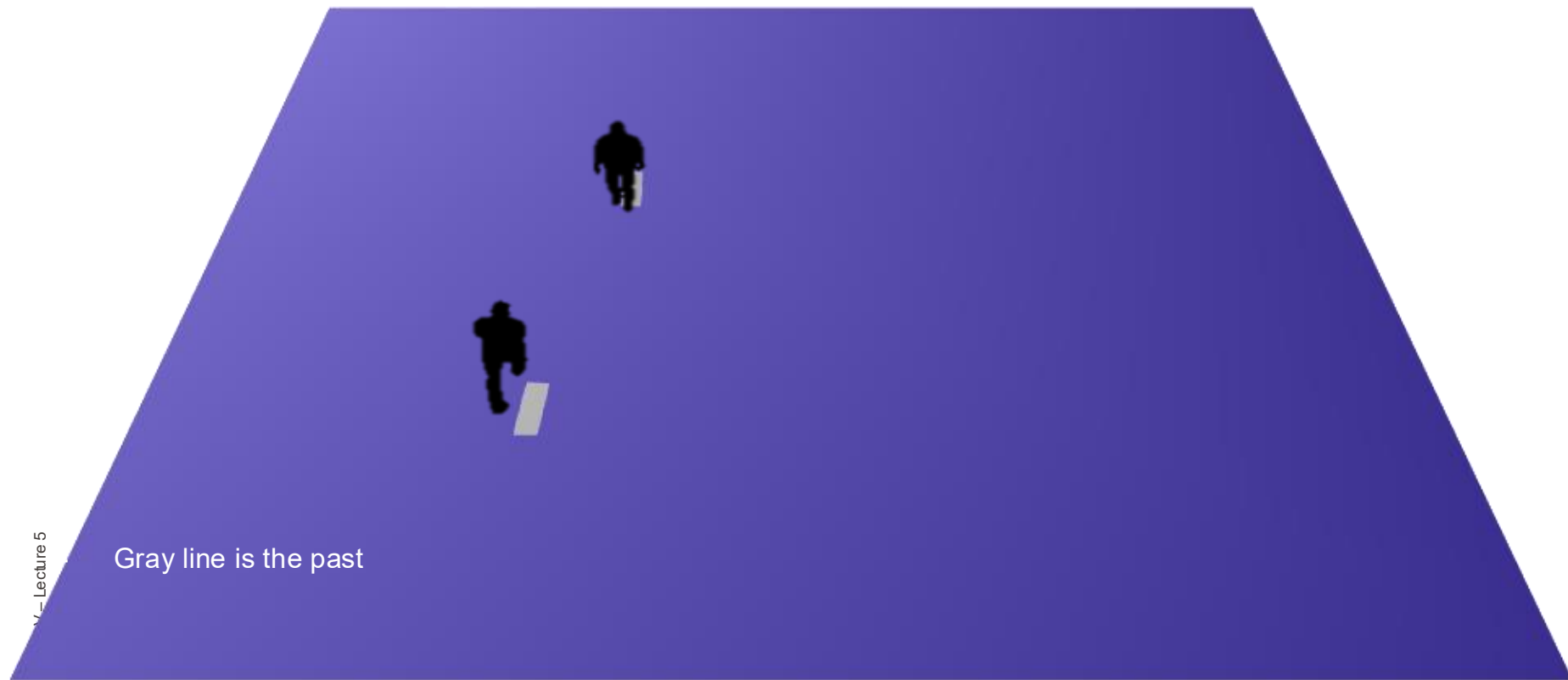


Gray line is the past





Gray line is the past





Gray line is the past

Can you predict the future?



Option 1

Gray line is the past

Can you predict the future?



Option 2

Gray line is the past
Black line is the future

Can you predict the future?



Option 3

Gray line is the past
Black line is the future

The future is...



Option 3 !

Gray line is the past
Black line is the future



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past







- Black line is the ground truth trajectory
- Gray line is the past



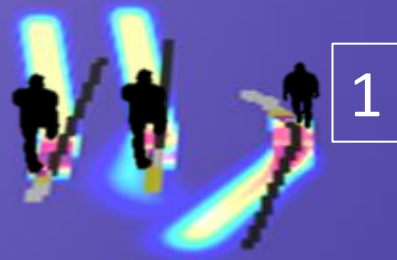
- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



Lecture 5

- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



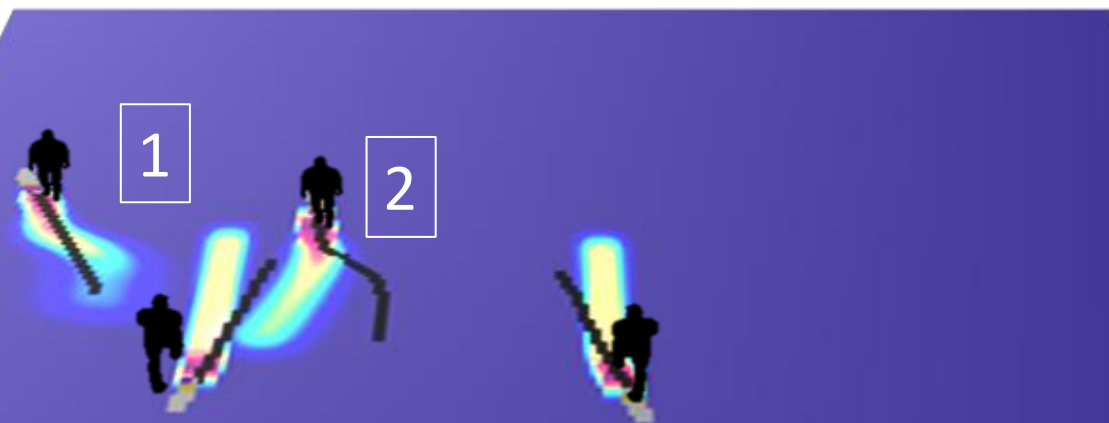
- Black line is the ground truth trajectory
- Gray line is the past



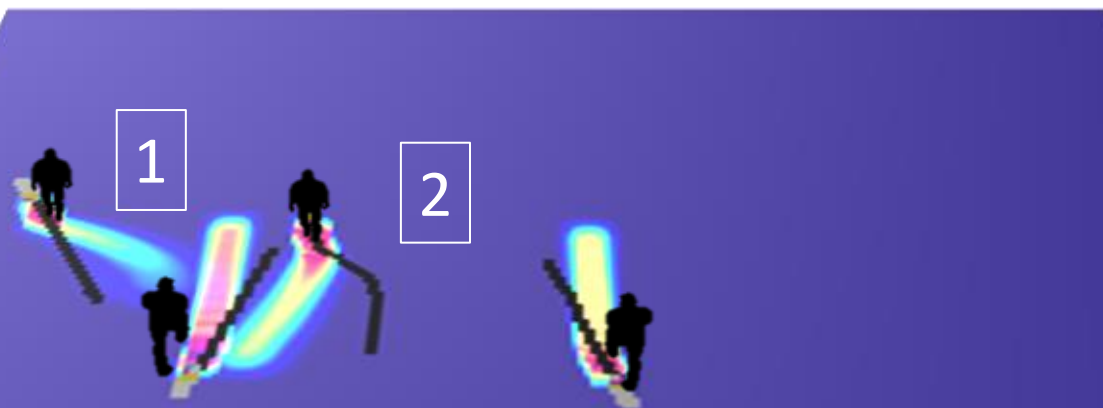
- Black line is the ground truth trajectory
- Gray line is the past



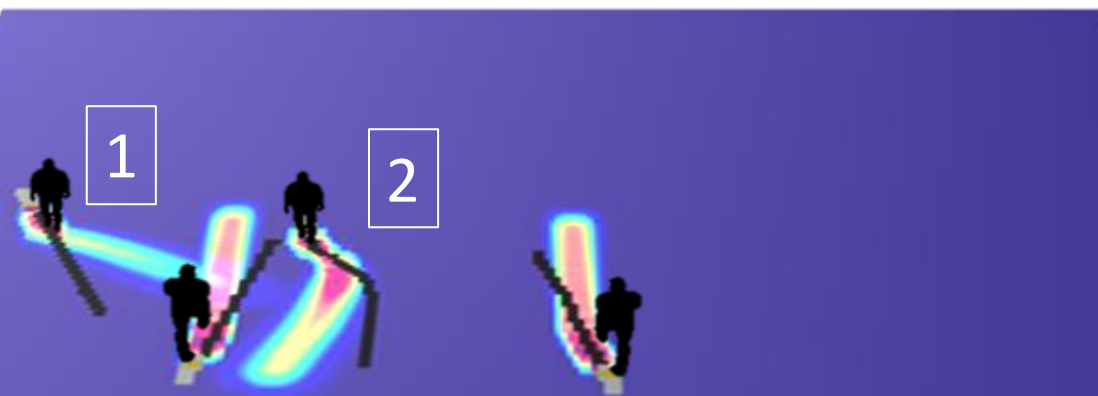
- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



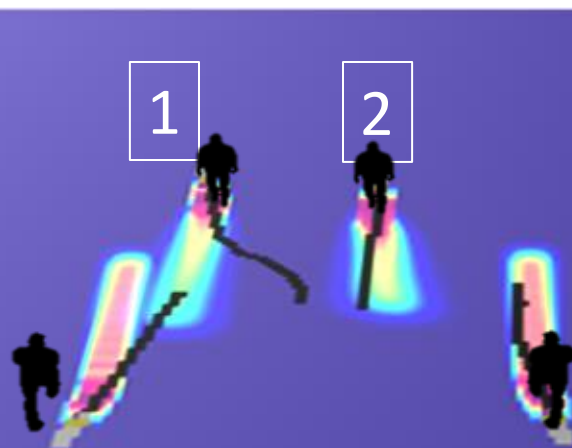
- Black line is the ground truth trajectory
- Gray line is the past



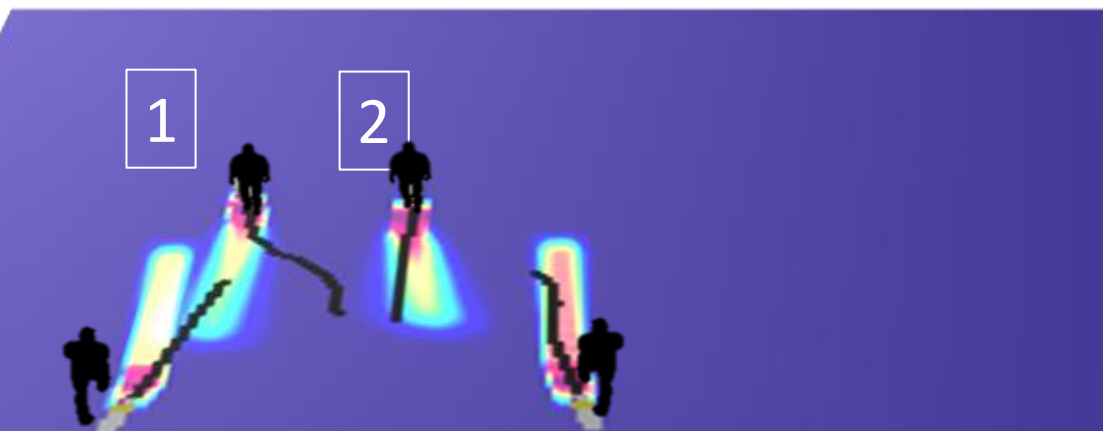
- Black line is the ground truth trajectory
- Gray line is the past



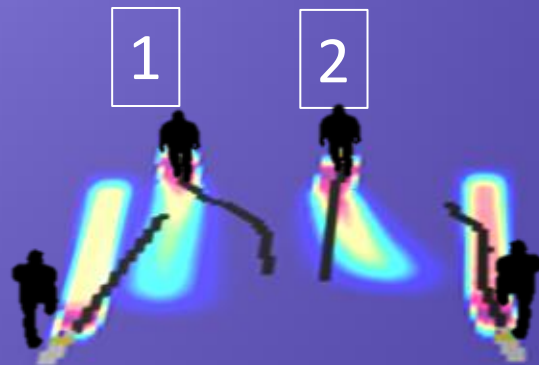
- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past



- Black line is the ground truth trajectory
- Gray line is the past

- Book: Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
(<http://www.deeplearningbook.org/>)

- Class on CNN:
<http://cs231n.stanford.edu/>

- Social LSTM: human trajectory prediction in crowded scene:
<https://infoscience.epfl.ch/record/230265?ln=fr>

Additional materials

Numpy VS pytorch

Numpy

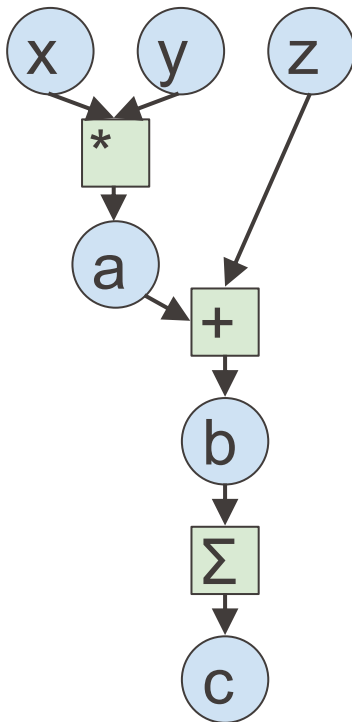
```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

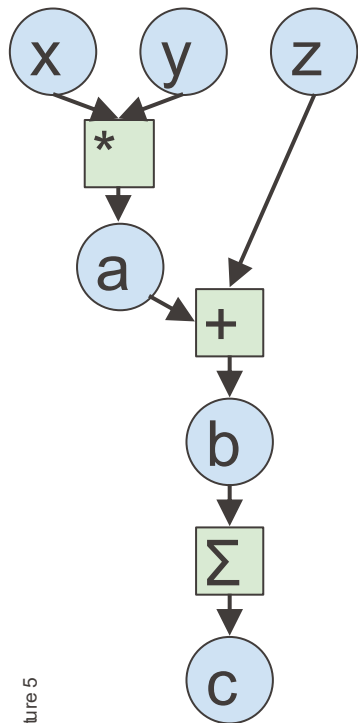
a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```



Problems:

- Can't run on GPU
- Have to compute our own gradients



PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

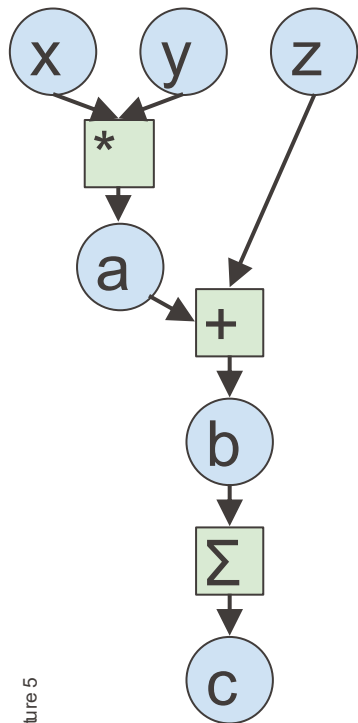
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

Computational Graphs



Define **Variables** to start building a computational graph

PyTorch

```
import torch
from torch.autograd import Variable

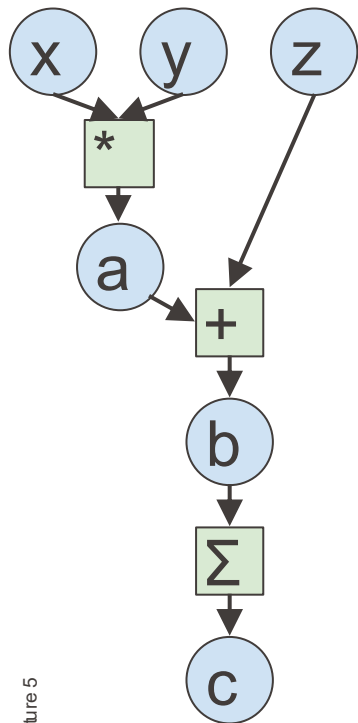
N, D = 3, 4
```

```
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)
```

```
a = x * y
b = a + z
c = torch.sum(b)
```

```
c.backward()
```

```
print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```



Forward pass
looks just like
numpy

PyTorch

```
import torch
from torch.autograd import Variable

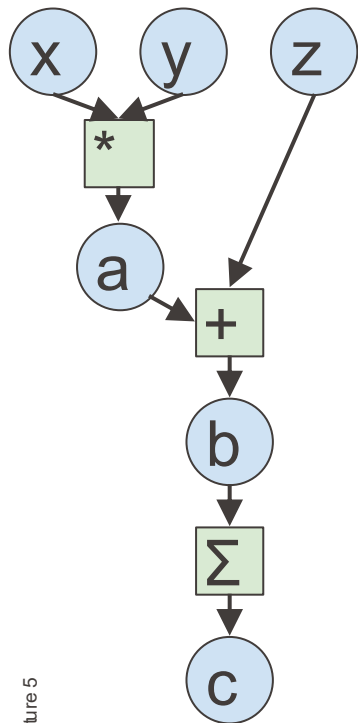
N, D = 3, 4

x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```



Calling `c.backward()`
computes all
gradients

PyTorch

```
import torch
from torch.autograd import Variable

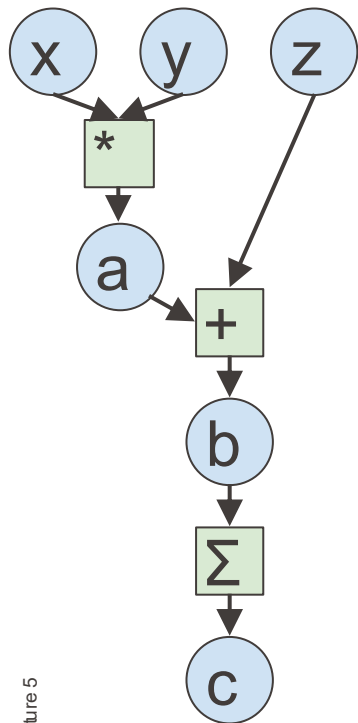
N, D = 3, 4

x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```



Run on GPU by
casting to `.cuda()`

PyTorch

A. Alahi

```

import torch
from torch.autograd import Variable

N, D = 3, 4

x = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
y = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
z = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)

```


Numpy

```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

x = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
y = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
z = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

PyTorch: Three Levels of Abstraction

Tensor: Imperative ndarray, but runs on GPU

Variable: Node in a computational graph; stores data and gradient

Module: A neural network layer; may store state or learnable weights

PyTorch Tensors are just like numpy arrays, but they can run on GPU.

No built-in notion of computational graph, or gradients, or deep learning.

Here we fit a two-layer net using PyTorch Tensors:

```
import torch
```

```
dtype = torch.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
```

```
x = torch.randn(N, D_in).type(dtype)
```

```
y = torch.randn(N, D_out).type(dtype)
```

```
w1 = torch.randn(D_in, H).type(dtype)
```

```
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    h = x.mm(w1)
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu.mm(w2)
```

```
    loss = (y_pred - y).pow(2).sum()
```

```
    grad_y_pred = 2.0 * (y_pred - y)
```

```
    grad_w2 = h_relu.t().mm(grad_y_pred)
```

```
    grad_h_relu = grad_y_pred.mm(w2.t())
```

```
    grad_h = grad_h_relu.clone()
```

```
    grad_h[h < 0] = 0
```

```
    grad_w1 = x.t().mm(grad_h)
```

```
    w1 -= learning_rate * grad_w1
```

```
    w2 -= learning_rate * grad_w2
```

Create random tensors
for data and weights



```
import torch

dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

Forward pass: compute predictions and loss



```
import torch

dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

Backward pass:
manually compute
gradients



```
import torch

dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

```
import torch

dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

Gradient descent
step on weights



To run on GPU, just cast tensors to a cuda datatype!

```
import torch
```

```
dtype = torch.cuda.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
```

```
x = torch.randn(N, D_in).type(dtype)
```

```
y = torch.randn(N, D_out).type(dtype)
```

```
w1 = torch.randn(D_in, H).type(dtype)
```

```
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    h = x.mm(w1)
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu.mm(w2)
```

```
    loss = (y_pred - y).pow(2).sum()
```

```
    grad_y_pred = 2.0 * (y_pred - y)
```

```
    grad_w2 = h_relu.t().mm(grad_y_pred)
```

```
    grad_h_relu = grad_y_pred.mm(w2.t())
```

```
    grad_h = grad_h_relu.clone()
```

```
    grad_h[h < 0] = 0
```

```
    grad_w1 = x.t().mm(grad_h)
```

```
w1 -= learning_rate * grad_w1
```

```
w2 -= learning_rate * grad_w2
```


A PyTorch **Variable** is a node in a computational graph

x.data is a Tensor

x.grad is a Variable of gradients
(same shape as x.data)

x.grad.data is a Tensor of gradients

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

PyTorch Tensors and Variables
have the same API!

Variables remember how they were
created (for backprop)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

We will not want gradients
(of loss) with respect to data

Do want gradients with
respect to weights

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

Forward pass looks exactly the same as the Tensor version, but everything is a variable now

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

Compute gradient of loss
with respect to w1 and w2
(zero out grads first)



```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

Make gradient
step on weights

PyTorch: New Autograd Functions

Define your own autograd functions by writing forward and backward for Tensors

(similar to modular layers in A2)

```
class ReLU(torch.autograd.Function):  
    def forward(self, x):  
        self.save_for_backward(x)  
        return x.clamp(min=0)  
  
    def backward(self, grad_y):  
        x, = self.saved_tensors  
        grad_input = grad_y.clone()  
        grad_input[x < 0] = 0  
        return grad_input
```

PyTorch: New Autograd Functions

```
class ReLU(torch.autograd.Function):  
    def forward(self, x):  
        self.save_for_backward(x)  
        return x.clamp(min=0)  
  
    def backward(self, grad_y):  
        x, = self.saved_tensors  
        grad_input = grad_y.clone()  
        grad_input[x < 0] = 0  
        return grad_input
```

```
N, D_in, H, D_out = 64, 1000, 100, 10  
  
x = Variable(torch.randn(N, D_in), requires_grad=False)  
y = Variable(torch.randn(N, D_out), requires_grad=False)  
w1 = Variable(torch.randn(D_in, H), requires_grad=True)  
w2 = Variable(torch.randn(H, D_out), requires_grad=True)  
  
learning_rate = 1e-6  
for t in range(500):  
    relu = ReLU()  
    y_pred = relu(x.mm(w1)).mm(w2)  
    loss = (y_pred - y).pow(2).sum()  
  
    if w1.grad: w1.grad.data.zero_()  
    if w2.grad: w2.grad.data.zero_()  
    loss.backward()  
  
    w1.data -= learning_rate * w1.grad.data  
    w2.data -= learning_rate * w2.grad.data
```

Can use our new autograd function in the forward pass

Higher-level wrapper for
working with neural nets

Similar to Keras and friends ...
but only one, and it's good =)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

Define our model as a
sequence of layers



nn also defines common
loss functions



```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

Forward pass: feed data
to model, and prediction
to loss function

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

Backward pass:
compute all gradients

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

Make gradient step on
each model parameter

Use an **optimizer** for different update rules

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
optimizer = torch.optim.Adam(model.parameters(),
                               lr=learning_rate)

for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    optimizer.zero_grad()
    loss.backward()

    optimizer.step()
```

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
optimizer = torch.optim.Adam(model.parameters(),
                               lr=learning_rate)

for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    optimizer.zero_grad()
    loss.backward()

    optimizer.step()
```

Update all parameters
after computing gradients

PyTorch: nn

Define new Modules

A PyTorch **Module** is a neural net layer; it inputs and outputs Variables

Modules can contain weights (as Variables) or other Modules

You can define your own Modules using autograd!

```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```


PyTorch: nn

Define new Modules

Define our whole model
as a single Module

```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

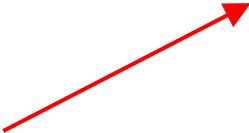
criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

PyTorch: nn

Define new Modules

Initializer sets up two children (Modules can contain modules)



```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

PyTorch: nn

Define new Modules

Define forward pass using
child modules and
autograd ops on Variables

No need to define
backward - autograd will
handle it

```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

PyTorch: nn

Define new Modules

Construct and train an instance of our model



```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

A **DataLoader** wraps a **Dataset** and provides minibatching, shuffling, multithreading, for you

When you need to load custom data, just write your own Dataset class

```
import torch
from torch.autograd import Variable
from torch.utils.data import TensorDataset, DataLoader

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

loader = DataLoader(TensorDataset(x, y), batch_size=8)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for epoch in range(10):
    for x_batch, y_batch in loader:
        x_var, y_var = Variable(x), Variable(y)
        y_pred = model(x_var)
        loss = criterion(y_pred, y_var)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Iterate over loader to form minibatches

Loader gives Tensors so you need to wrap in Variables

```
import torch
from torch.autograd import Variable
from torch.utils.data import TensorDataset, DataLoader

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

loader = DataLoader(TensorDataset(x, y), batch_size=8)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for epoch in range(10):
    for x_batch, y_batch in loader:
        x_var, y_var = Variable(x), Variable(y)
        y_pred = model(x_var)
        loss = criterion(y_pred, y_var)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

PyTorch: Pretrained Models

Super easy to use pretrained models with torchvision

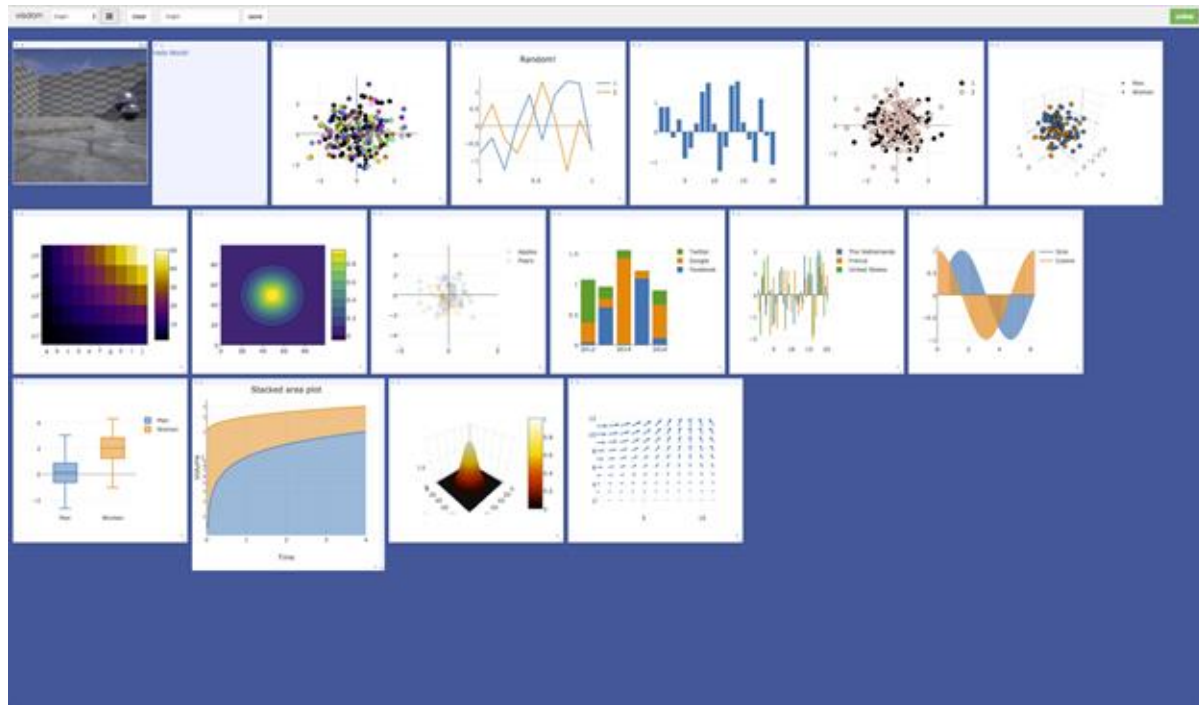
<https://github.com/pytorch/vision>

```
import torch
import torchvision

alexnet = torchvision.models.alexnet(pretrained=True)
vgg16 = torchvision.models.vgg16(pretrained=True)
resnet101 = torchvision.models.resnet101(pretrained=True)
```


Somewhat similar to
TensorBoard: add logging
to your code, then
visualized in a browser

Can't visualize
computational graph
structure (yet?)



This image is licensed under [CC-BY 4.0](https://creativecommons.org/licenses/by/4.0/); no changes were made to the image

min-char-rnn.py gist: 112 lines of Python

```

1  """
2  Minimal character-level vanilla RNN model. Written by Andrej Karpathy (@karpathy)
3  BSD license
4  """
5  import numpy as np
6
7  # data I/O
8
9  data = open('input.txt', 'r').read() # should be simple plain text file
10 chars = list(set(data))
11 data_size, vocab_size = len(data), len(chars)
12 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
13 char_to_ix = { ch:i for i,ch in enumerate(chars) }
14 ix_to_char = { ix:i for i,ix in enumerate(chars) }
15
16 # hyperparameters
17 hidden_size = 100 # size of hidden layer of neurons
18 seq_length = 25 # number of steps to unroll the RNN for
19 learning_rate = 1e-1
20
21 # model parameters
22 wih = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
23 whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
24 why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
25 bi = np.zeros((hidden_size, 1)) # hidden bias
26 by = np.zeros((vocab_size, 1)) # output bias
27
28 def lossfun(inputs, targets, hprev):
29     """
30     inputs, targets are both list of integers.
31     hprev is Nx1 array of initial hidden state
32     returns the loss, gradients on model parameters, and last hidden state
33     """
34     xs, hs, ys, ps = [], [], [], []
35     hs[-1] = np.copy(hprev)
36     loss = 0
37     # forward pass
38     for t in xrange(len(inputs)):
39         xt = np.zeros((vocab_size, 1)) # encode in 1-of-K representation
40         xt[t][inputs[t]] = 1
41         ht = np.tanh(np.dot(wih, xt[t]) + np.dot(whh, hs[t-1]) + bi) # hidden state
42         yt = np.dot(why, ht[t]) + by # unnormalized log probabilities for next char
43         pt = np.exp(yt) / np.sum(np.exp(yt)) # probabilities for next char
44         loss += -np.log(pt)[targets[t,0]] # softmax (cross-entropy) loss
45     # backward pass: compute gradients going backwards
46     dht, dwh, dby = np.zeros_like(wih), np.zeros_like(whh), np.zeros_like(why)
47     dht, dby = np.zeros_like(bi), np.zeros_like(by)
48     dnext = np.zeros_like(hs[0])
49     for t in reversed(xrange(len(inputs))):
50         dy = np.copy(pt[t])
51         dy[targets[t]] = 1 # backward loss y
52         dht += np.dot(dy, hs[t].T)
53         dby += dy
54         dh = np.dot(why.T, dy) + dnext # backprop into h
55         dxtw = (1 - ht[t]**2) * dh # backprop through tanh nonlinearity
56         dht += dxtw
57         dwh += np.dot(dxtw, xt[t].T)
58         dnext = np.dot(whh.T, dhtw)
59     for dparam in [dwh, dwh, dby, dht, dby]:
60         np.clip(dparam, -1, 1, out=dparam) # clip to mitigate exploding gradients
61     return loss, dwh, dwh, dby, dht, dby, hs[len(inputs)-1]
62
63 def sample(h, seed_ix, n):
64     """
65     sample a sequence of integers from the model
66     h is memory state, seed_ix is seed letter for first time step
67     """
68     x = np.zeros((vocab_size, 1))
69     x[seed_ix] = 1
70     seqs = []
71     for t in xrange(n):
72         h = np.tanh(np.dot(wih, x) + np.dot(whh, h) + bi)
73         y = np.dot(why, h) + by
74         p = np.exp(y) / np.sum(np.exp(y))
75         ix = np.random.choice(range(vocab_size), p=p.ravel())
76         x = np.zeros((vocab_size, 1))
77         x[ix] = 1
78         seqs.append(ix)
79     return seqs
80
81 N, p, q, R =
82 dwh, dwh, dby = np.zeros_like(wih), np.zeros_like(whh), np.zeros_like(why)
83 dht, dby = np.zeros_like(bi), np.zeros_like(by) # memory variables for adjoint
84 smooth_loss = -np.log(1.0/vocab_size)/seq_length # loss at iteration 0
85 while True:
86     # prepare inputs (we're sampling from left to right in steps seq_length long)
87     if p+seq_length > len(data) or n == 0:
88         hprev = np.zeros((hidden_size,1)) # reset RNN memory
89         p = 0 # go from start of data
90         inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
91         targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
92
93     # sample from the model! now and then
94     if n % 100 == 0:
95         sample_ix = sample(hprev, inputs[0], 200)
96         txt = "".join(ix_to_char[ix] for ix in sample_ix)
97         print "----in %s %s----" % (txt, p)
98
99     # forward seq_length characters through the net and fetch gradient
100     loss, dwh, dwh, dby, dht, dby, hprev = lossfun(inputs, targets, hprev)
101     smooth_loss = smooth_loss * 0.999 + loss * 0.001
102     if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
103
104     # perform parameter update with Adagrad
105     for param, dparam, sss in zip([wih, whh, why, bi, by],
106                                   [dwh, dwh, dby, dht, dby],
107                                   [dwh, dwh, dby, dht, dby]):
108         sss += dparam * dparam
109         param -= learning_rate * dparam / np.sqrt(sss + 1e-4) # Adagrad update
110
111     p += seq_length # move data pointer
112     n += 1 # iteration counter

```

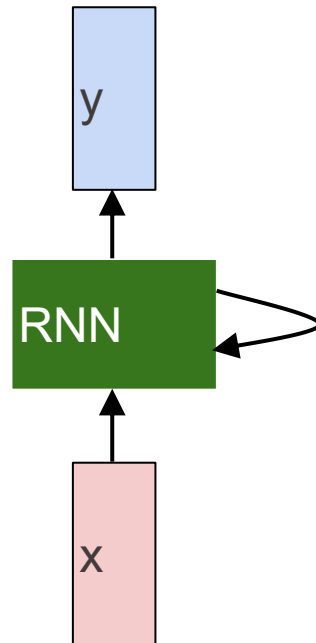
<https://gist.github.com/karpathy/44dee566867f8291f086>

THE SONNETS

by William Shakespeare

From fairest creatures we desire increase,
That thereby beauty's rose might never die,
But as the ripper should by time decease,
His tender heir might bear his memory:
But thou, contracted to thine own bright eyes,
Feed'st thy light's flame with self-substantial fuel,
Making a famine where abundance lies,
Thyself thy foe, to thy sweet self too cruel:
Thou that art now the world's fresh ornament,
And only herald to the gaudy spring,
Within thine own bud buriest thy content,
And tender churl mak'st waste in niggarding:
Pity the world, or else this glutton be,
To eat the world's due, by the grave and thee.

When forty winters shall besiege thy brow,
And dig deep trenches in thy beauty's field,
Thy youth's proud livery so gazed on now,
Will be a tatter'd weed of small worth held:
Then being asked, where all thy beauty lies,
Where all the treasure of thy lusty days;
To say, within thine own deep sunken eyes,
Were an all-eating shame, and thriftless praise.
How much more praise deserv'd thy beauty's use,
If thou couldst answer 'This fair child of mine
Shall sum my count, and make my old excuse,'
Proving his beauty by succession thine!
This were to be new made when thou art old,
And see thy blood warm when thou feel'st it cold.



tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

↓ train more

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

↓ train more

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and offer.

↓ train more

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

PANDARUS:

Alas, I think he shall be come approached and the day
When little strain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and
my fair nudes begun out of the fact, to be conveyed,
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

VIOLA:

Why, Salisbury must find his flesh and thought
That which I am not apt, not a man and in fire,
To show the reining of the raven and the wars
To grace my hand reproach within, and not a fair are hand,
That Caesar and my goodly father's world;
When I was heaven of presence and our fleets,
We spare with hours, but cut thy council I am great,
Murdered and by thy master's ready there
My power to give thee but so much as hell:
Some service in the noble bondman here,
Would show him to her wine.

KING LEAR:

O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.

EPFL The Stacks Project: open source algebraic geometry textbook

 **The Stacks Project**

[home](#) [about](#) [tags explained](#) [tag lookup](#) [browse](#) [search](#) [bibliography](#) [recent comments](#) [blog](#) [add slogans](#)

Browse chapters

Part	Chapter	online	TeX source	view pdf
Preliminaries				
	1. Introduction	online	tex 	pdf 
	2. Conventions	online	tex 	pdf 
	3. Set Theory	online	tex 	pdf 
	4. Categories	online	tex 	pdf 
	5. Topology	online	tex 	pdf 
	6. Sheaves on Spaces	online	tex 	pdf 
	7. Sites and Sheaves	online	tex 	pdf 
	8. Stacks	online	tex 	pdf 
	9. Fields	online	tex 	pdf 
	10. Commutative Algebra	online	tex 	pdf 

Parts

- [Preliminaries](#)
- [Schemes](#)
- [Topics in Scheme Theory](#)
- [Algebraic Spaces](#)
- [Topics in Geometry](#)
- [Deformation Theory](#)
- [Algebraic Stacks](#)
- [Miscellany](#)

Statistics

The Stacks project now consists of

- 455910 lines of code
- 14221 tags (56 inactive tags)
- 2366 sections

Latex source

<http://stacks.math.columbia.edu/>

The stacks project is licensed under the [GNU Free Documentation License](#)

For $\bigoplus_{n=1, \dots, m}$ where $\mathcal{L}_{m,*} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparico in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{fppf} and $U \rightarrow U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ?? . Hence we obtain a scheme S and any open subset $W \subset U$ in $\text{Sh}(G)$ such that $\text{Spec}(R') \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X,x}$ is a scheme where $x, x', s'' \in S'$ such that $\mathcal{O}_{X,x'} \rightarrow \mathcal{O}'_{X',x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\text{GL}_{S'}(x'/S'')$ and we win. $\square$

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and T_i is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\tilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\text{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{fppf}^{\text{opp}}, (\text{Sch}/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \mapsto (U, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result for prove any open covering follows from the less of Example ?? . It may replace S by $X_{\text{spaces}, \text{étale}}$ which gives an open subspace of X and T equal to S_{Zar} , see Descent, Lemma ?? . Namely, by Lemma ?? we see that R is geometrically regular over S .

Lemma 0.1. Assume (3) and (3) by the construction in the description.

Suppose $X = \lim |X|$ (by the formal open covering X and a single map $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_{X, \mathcal{O}_X}).$$

When in this case of to show that $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$ is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U in X' is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1) f is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \coprod_{i=1, \dots, n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \lim_i X_i$. $\square$

The following lemma surjective restrocomposes of this implies that $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{X, \dots, 0}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $\mathcal{I} = \mathcal{I}_1 \subset \mathcal{I}'_n$. Since $\mathcal{I}^n \subset \mathcal{I}^n$ are nonzero over $i_0 \leq \mathfrak{p}$ is a subset of $\mathcal{I}_{n,0} \circ \bar{A}_2$ works.

Lemma 0.3. In Situation ?? . Hence we may assume $\mathfrak{q}' = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (??). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Proof. Omitted. □

Lemma 0.1. Let $\mathcal{C}$ be a set of the construction.

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules. □

Lemma 0.2. This is an integer $\mathbb{Z}$ is injective.

Proof. See Spaces, Lemma ?? □

Lemma 0.3. Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset \mathcal{X}$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b : X \rightarrow Y' \rightarrow Y \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(U)$ which is locally of finite type. □

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram

$$\begin{array}{ccc} S & \xrightarrow{\quad} & \\ \downarrow & & \\ \xi & \xrightarrow{\quad} & \mathcal{O}_{X'} \\ \text{gor}_s \uparrow & & \searrow \\ & & \\ & \alpha' \xrightarrow{\quad} & \\ \downarrow & & \\ & \alpha' \xrightarrow{\quad} & \alpha \end{array} \quad \begin{array}{c} X \\ \downarrow \\ \text{Mor}_{\text{Sets}} \\ \text{d}(\mathcal{O}_{X_{X/\mathbb{A}}}, \mathcal{G}) \end{array}$$

is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_* . This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

□

Proof. We have see that $X = \text{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U . □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.

A reduced above we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a “field

$$\mathcal{O}_{X,x} \xrightarrow{\quad} \mathcal{F}_x^{-1}(\mathcal{O}_{X_{\text{étale}}}) \xrightarrow{\quad} \mathcal{O}_{X_t}^{-1} \mathcal{O}_{X_\lambda}(\mathcal{O}_{X_0}^{\vee})$$

is an isomorphism of covering of $\mathcal{O}_{X_t}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.

The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$ -algebra with $\mathcal{F}$ are opens of finite type over S .

If $\mathcal{F}$ is a scheme theoretic image points. □

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_\lambda}$ is a closed immersion, see Lemma ?? . This is a sequence of $\mathcal{F}$ is a similar morphism.



This repository Search

Explore Gist Blog Help



karpathy



torvalds / linux

Watch 3,711

★ Star 23,054

Fork 9,141

Linux kernel source tree

520,037 commits

1 branch

420 releases

5,039 contributors



branch: master

linux / +



Merge branch 'drm-fixes' of git://people.freedesktop.org/~airlied/linux



torvalds authored 9 hours ago

latest commit 4b1706927d



Documentation

Merge git://git.kernel.org/pub/scm/linux/kernel/git/nab/target-pending

6 days ago



arch

Merge branch 'x86-urgent-for-linus' of git://git.kernel.org/pub/scm/lin...

a day ago



block

block: discard bdi_unregister() in favour of bdi_destroy()

9 days ago



crypto

Merge git://git.kernel.org/pub/scm/linux/kernel/git/herbert/crypto-2.6

10 days ago



drivers

Merge branch 'drm-fixes' of git://people.freedesktop.org/~airlied/linux

9 hours ago



firmware

firmware/ihex2fw.c: restore missing default in switch statement

2 months ago



fs

vfs: read file_handle only once in handle_to_path

4 days ago



include

Merge branch 'perf-urgent-for-linus' of git://git.kernel.org/pub/scm/lin...

a day ago



init

init: fix regression by supporting devices with major:minor:offset fo...

a month ago



lib

Merge branch 'for-linus' of git://git.kernel.org/pub/scm/linux/kernel...

a month ago



Code



Pull requests

74



Pulse



Graphs

HTTPS clone URL

https://github.c

You can clone with [HTTPS](#), [SSH](#), or [Subversion](#).

Clone in Desktop



Download ZIP

```

static void do_command(struct seq_file *m, void *v)
{
    int column = 32 << (cmd[2] & 0x80);
    if (state)
        cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
    else
        seq = 1;
    for (i = 0; i < 16; i++) {
        if (k & (1 << i))
            pipe = (in_use & UMXTHREAD_UNCCA) +
                ((count & 0x00000000ffffffff) & 0x0000000f) << 8;
        if (count == 0)
            sub(pid, ppc_md.kexec_handle, 0x20000000);
        pipe_set_bytes(i, 0);
    }
    /* Free our user pages pointer to place camera if all dash */
    subsystem_info = &of_changes[PAGE_SIZE];
    rek_controls(offset, idx, &soffset);
    /* Now we want to deliberately put it to device */
    control_check_polarity(&context, val, 0);
    for (i = 0; i < COUNTER; i++)
        seq_puts(s, "policy ");
}

```

Generated C code

```
/*  
 * Copyright (c) 2006-2010, Intel Mobile Communications. All rights reserved.  
 *  
 * This program is free software; you can redistribute it and/or modify it  
 * under the terms of the GNU General Public License version 2 as published by  
 * the Free Software Foundation.  
 *  
 * This program is distributed in the hope that it will be useful,  
 * but WITHOUT ANY WARRANTY; without even the implied warranty of  
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the  
 *  
 * GNU General Public License for more details.  
 *  
 * You should have received a copy of the GNU General Public License  
 * along with this program; if not, write to the Free Software Foundation,  
 * Inc., 675 Mass Ave, Cambridge, MA 02139, USA.  
 */  
  
#include <linux/kexec.h>  
#include <linux/errno.h>  
#include <linux/io.h>  
#include <linux/platform_device.h>  
#include <linux/multi.h>  
#include <linux/ckevent.h>  
  
#include <asm/io.h>  
#include <asm/prom.h>  
#include <asm/e820.h>  
#include <asm/system_info.h>  
#include <asm/setew.h>  
#include <asm/pgproto.h>
```

```

#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>

#define REG_PG    vesa_slot_addr_pack
#define PFM_NOCOMP AFSR(0, load)
#define STACK_DDR(type)      (func)

#define SWAP_ALLOCATE(nr)      (e)
#define emulate_sigs() arch_get_unaligned_child()
#define access_rw(TST) asm volatile("movd %!esp, %0, %!3" : : "r" (0)); \
    if (__type & DO_READ)

static void stat_PC_SEC __read_mostly offsetof(struct seq_argsqueue, \
    pC>[1]);

static void
os_prefix(unsigned long sys)
{
#ifdef CONFIG_PREEMPT
    PUT_PARAM_RAID(2, sel) = get_state_state();
    set_pid_sum((unsigned long)state, current_state_str(),
        (unsigned long)-1->lr_full; low;
}

```